（请大家预习）

第9章第2讲

数据聚类

Clustering Methods

张 燕 明

ymzhang@nlpr.ia.ac.cn

people.ucas.ac.cn/~ymzhang

模式分析与学习课题组(PAL)

多模态人工智能系统实验室 中科院自动化所

助教：杨 奇 (yangqi2021@ia.ac.cn)

张 涛 (zhangtao2021@ia.ac.cn)

上讲回顾

- **聚类任务**
  - 给定样本集合 $X=\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N\}$和度量样本间相似度(或者距离)的标准。聚类方法的输出是关于样本集 X 的一个划分，即$D = \{D_1, D_2, \ldots, D_K\}$。其中，$D_k$ (k=1,2,…,K)是 X 的一个子集，且满足：
    - $D_1 \cup D_2 \cup \cdots \cup D_K = X$
    - $D_i \cap D_j = \emptyset, \quad i \neq j$

上讲回顾

- **聚类方法分类**
  - **按照度量准则**
    - 基于**距离**的聚类方法：基于各种不同的距离或者相似性来度量点对之间的关系，如*K*-means，层次聚类等。
    - 基于**密度**的聚类方法：采用密度函数对样本进行描述，并得到聚类结果，如GMM, mean-shift, DBSCAN等。
    - 基于**连通性**的聚类方法：主要包含基于图的方法。高度连通的数据通常被聚为一簇，如谱聚类等。

上讲回顾

- *K*-means
  - Loss function

$$L(\mathbf{X}, \mathbf{Z}, \boldsymbol{\mu}) = \sum_{n=1}^{N}\sum_{k=1}^{K} z_{nk} \left\| \mathbf{x}_n - \boldsymbol{\mu}_k \right\|^2$$

  - *K*-means is a heuristic algorithm that optimizes $L(\mathbf{X}, \mathbf{Z}, \boldsymbol{\mu})$ w.r.t. $\mathbf{Z}$ and $\boldsymbol{\mu}$ by alternating between two steps:
    - Fix $\boldsymbol{\mu}$, minimize L w.r.t. $\mathbf{Z}$

$$\mathbf{z}_{nj} = \begin{cases} 1, & j = \arg\min_{k=\{1,\ldots,K\}} \left\| \mathbf{x}_n - \boldsymbol{\mu}_k \right\| \\ 0, & \text{otherwise} \end{cases}$$

    - Fix $\mathbf{Z}$, minimize L w.r.t. $\boldsymbol{\mu}$

$$\boldsymbol{\mu}_k = \frac{\sum_{n=1}^{N} I\{\mathbf{z}_{nk} == 1\}\mathbf{x}_n}{\sum_{n=1}^{N} I\{\mathbf{z}_{nk} == 1\}}$$

上讲回顾

- GMM
  - Density function

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

  - EM algorithm
    - E step: Fix (π, $\boldsymbol{\mu}$, Σ), minimize L w.r.t. $\mathbf{Z}$

$$z_{nk} = \frac{\pi_k N(\mathbf{x}\,|\,\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j N(\mathbf{x}\,|\,\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

    - M step: Fix $\mathbf{Z}$, minimize L w.r.t. (π, $\boldsymbol{\mu}$, Σ)

$$\pi_k = \frac{1}{N}\sum_{n=1}^{n} z_{nk}, \quad \boldsymbol{\mu}_k = \frac{\sum_{n=1}^{N} z_{nk}\mathbf{x}_n}{\sum_{n=1}^{N} z_{nk}}, \quad \boldsymbol{\Sigma}_k = \frac{\sum_{n=1}^{N} z_{nk}(\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^T}{\sum_{n=1}^{N} z_{nk}}$$

  - Relation between k-means and GMM
    - 当$\boldsymbol{\Sigma}_k = \sigma^2 \boldsymbol{I}$ 且$\sigma^2 \to 0$时，GMM中的E步趋近于最近邻分配，M步中均值更新趋近于样本均值，即GMM等价于K-means

Hierarchical Clustering
分级聚类

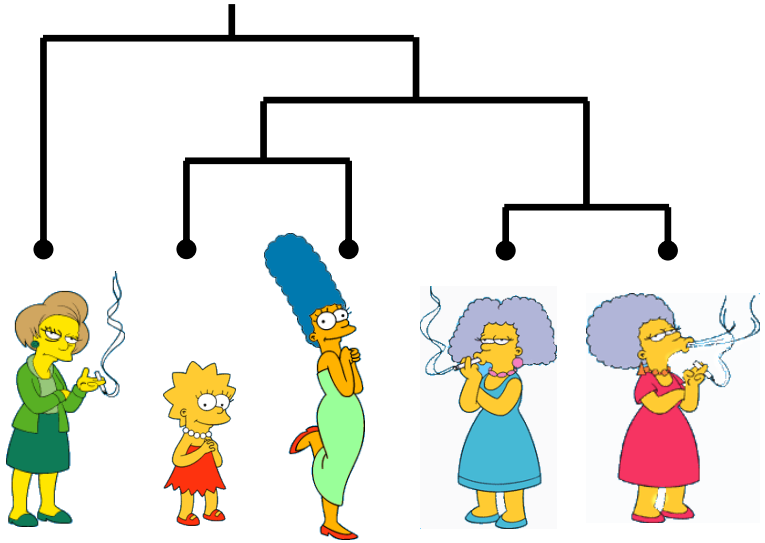
分级聚类(Hierarchical Clustering)

- **生物学上的物种分类**
  - 门、纲、目、科、属、种
  - 最相似的物种被归为同一个“种”
  - 这种分层次归类方法对生物学研究发挥着巨大的作用
    - 生物学意义
    - 生物学研究意义：解决分岐、发现新的物种。
  - 这种思想也可以自然地应用到聚类分析之中，称为**分级聚类、层次聚类**或者**系统聚类**。

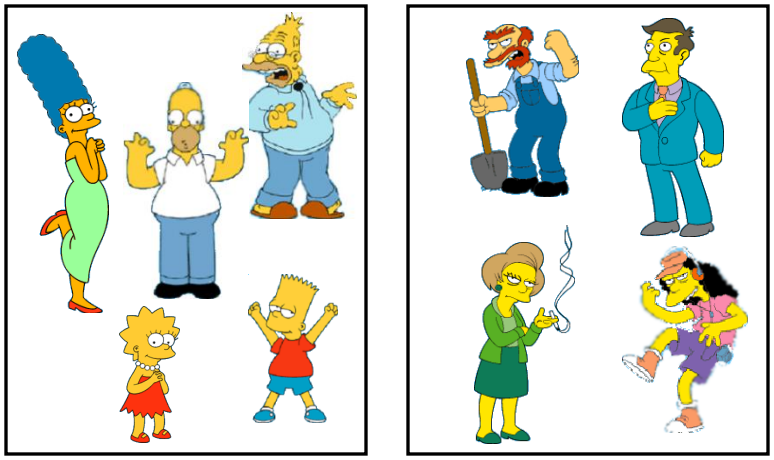
Two Types of Clustering

- **Partitional algorithms:** Construct various partitions and then evaluate them by some criterion
- **Hierarchical algorithms:** Create a hierarchical decomposition of the set of objects using some criterion

Hierarchical



Partitional



分级聚类

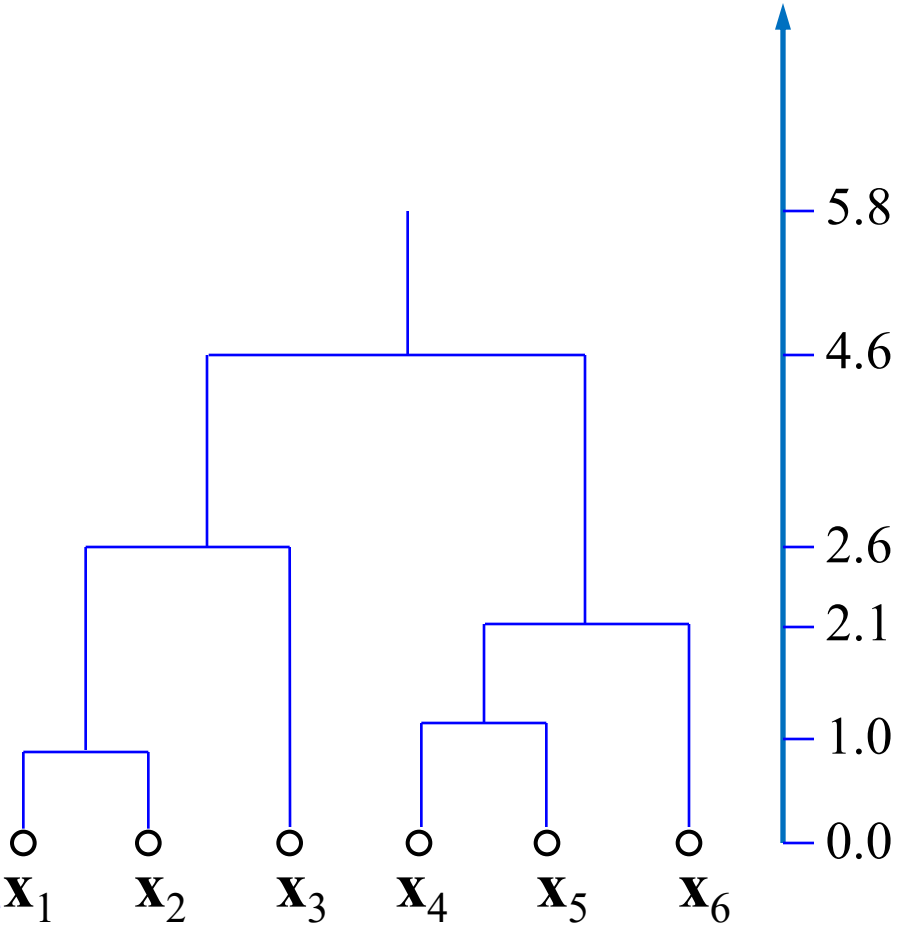
- **分级聚类方法**
  - 对于 n 个样本，极端的情况下，最多可以将数据分成 n 类；最少可以只分成一类，即全部样本都归为一类。
  - **凝聚的层次聚类（自底向上）**
    - 将每个样本作为一个类，然后根据给定的规则**逐渐合并**一些类，形成越来越大的类，直到某个终止条件得到满足。
  - **分裂的层次聚类（自顶向下）**
    - 将所有样本作为一个类，然后根据给定的规则**逐渐拆分**一些类，形成越来越小的类，直到某个终止条件得到满足。

分级聚类

- **自底向上的分级聚类步骤**
  - (1) 初始化：每个样本形成一个类
  - (2) 合并：计算任意两个类之间的距离（或相似性），将距离最小（或相似性最大）的两个类合并为一个类，其余类不变。
  - (3) 重复步骤 (2)：直到所有样本被合并到K个类之中。

分级聚类

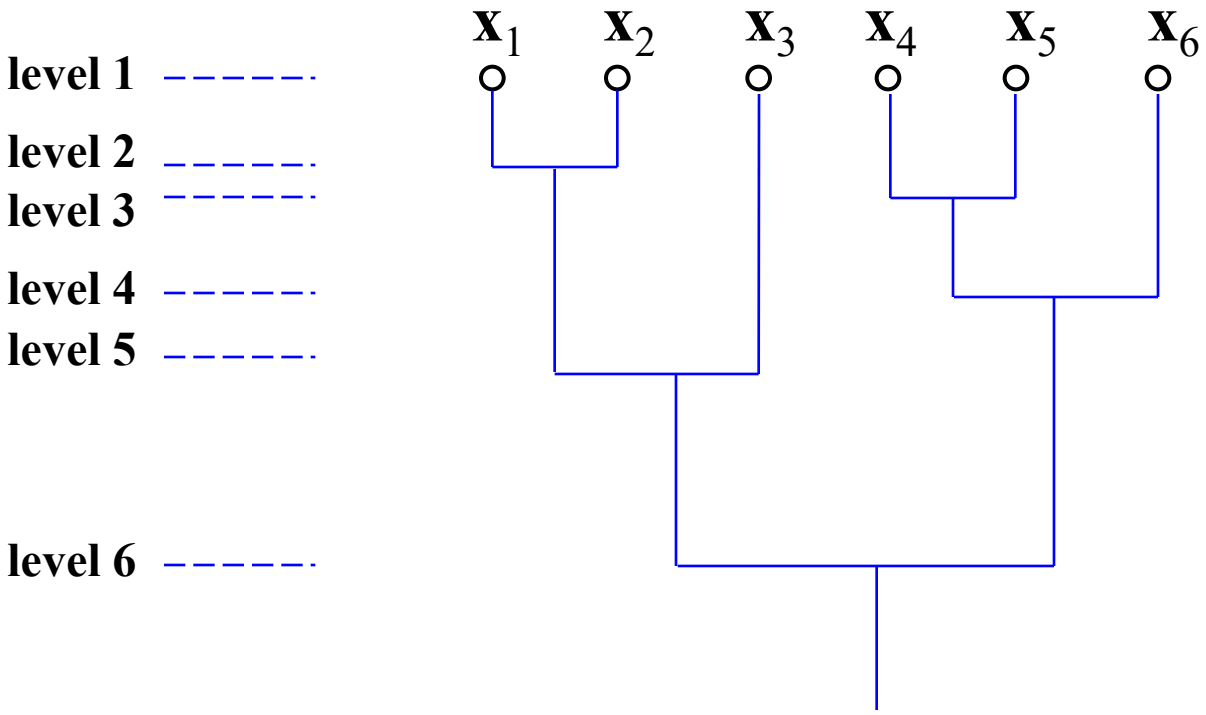
- **聚类树/系统树**
  - 用一棵树来描述分级聚类结果，称为**聚类树**(dendrogram)，或**系统树图**。
  - 如右图，最底层的每个节点表示一个样本，**采用树枝连接两个合并的样本**，树枝的长度反映两个节点之间的距离（或相似性）。
  - 为方便，**采用“水平”来表示分级聚类过程的不同阶段。**开始为水平1，每个样本为一类，然后每执行一次“合并”增加一个水平。



系统树图（采用距离）

分级聚类

- 系统树：分级聚类的另一种表示



系统树图（采用相似性）

分级聚类

- **分级聚类两个核心问题：**
  - 如何度量**样本之间**的距离（或相似性）
  - 如何度量**两个簇之间**的距离（或相似性）
- **样本之间的距离（或相似性）：**
  - 欧氏距离、马式距离、城区距离、匹配距离、…
  - 相关系数、高斯相似性函数、余弦、距离倒数、…

分级聚类

- 簇与簇之间的距离

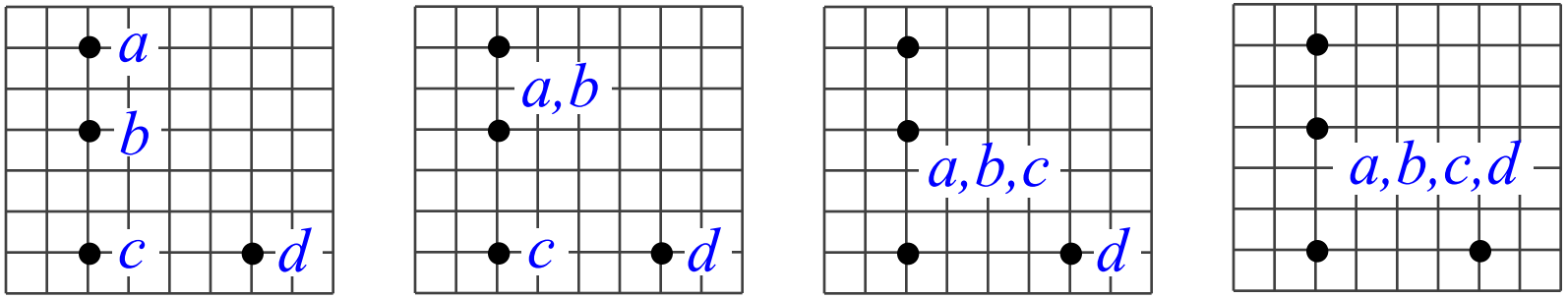
$d_{\min}\left(D_i, D_j\right) = \min_{\substack{\mathbf{x}\in D_i \\ \mathbf{x}'\in D_j}} \left\|\mathbf{x}-\mathbf{x}'\right\|$	最小距离
$d_{\max}\left(D_i, D_j\right) = \max_{\substack{\mathbf{x}\in D_i \\ \mathbf{x}'\in D_j}} \left\|\mathbf{x}-\mathbf{x}'\right\|$	最大距离
$d_{avg}\left(D_i, D_j\right) = \frac{1}{\left\|D_i\right\|\left\|D_j\right\|} \sum_{\mathbf{x}\in D_i} \sum_{\mathbf{x}'\in D_j} \left\|\mathbf{x}-\mathbf{x}'\right\|$	平均距离
$d_{mean}\left(D_i, D_j\right) = \left\|\mathbf{m}_i - \mathbf{m}_j\right\|$	中心距离

分级聚类

- **实践**
  - 根据数据特性、聚类目标的不同，通常需要采用不同的簇间距离。
  - 对同一数据集，采用不同的簇间距离通常会得到不同的聚类结果。

- **例子1：采用最近距离连接簇**

(注：a到d之间的距离取整数）



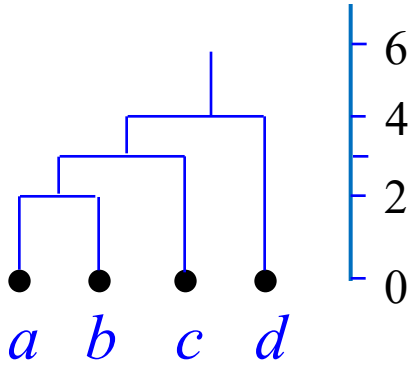
	b	c	d
a	2	5	6
b		3	5
c			4

→

	c	d
a,b	3	5
c		4

→

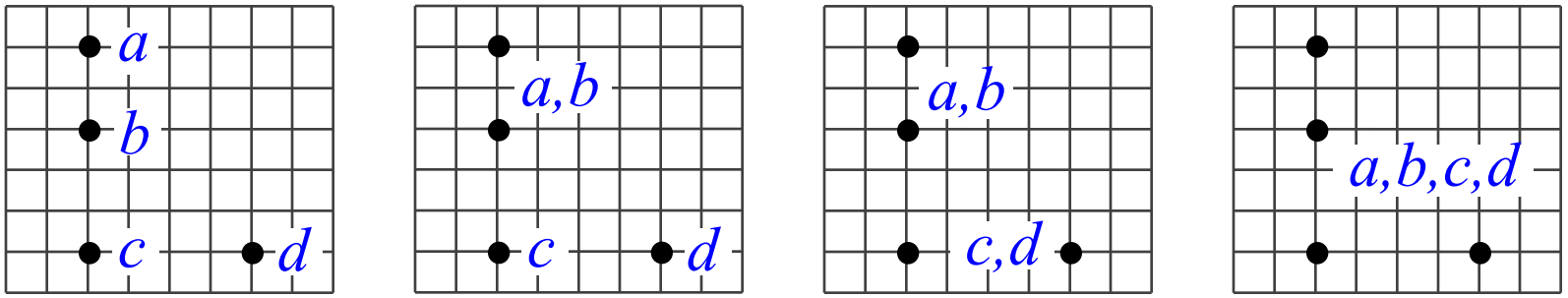
	d
a,b,c	4



簇与簇之间的距离采用最近距离

- **例子2：采用最远距离连接簇 (最大最小准则)**

(注：a到d之间的距离取整数)



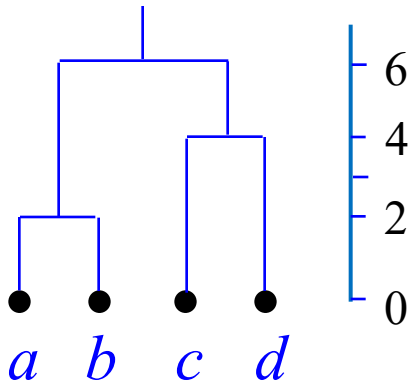
	b	c	d
a	2	5	6
b		3	5
c			4

→

	c	d
a,b	5	6
c		4

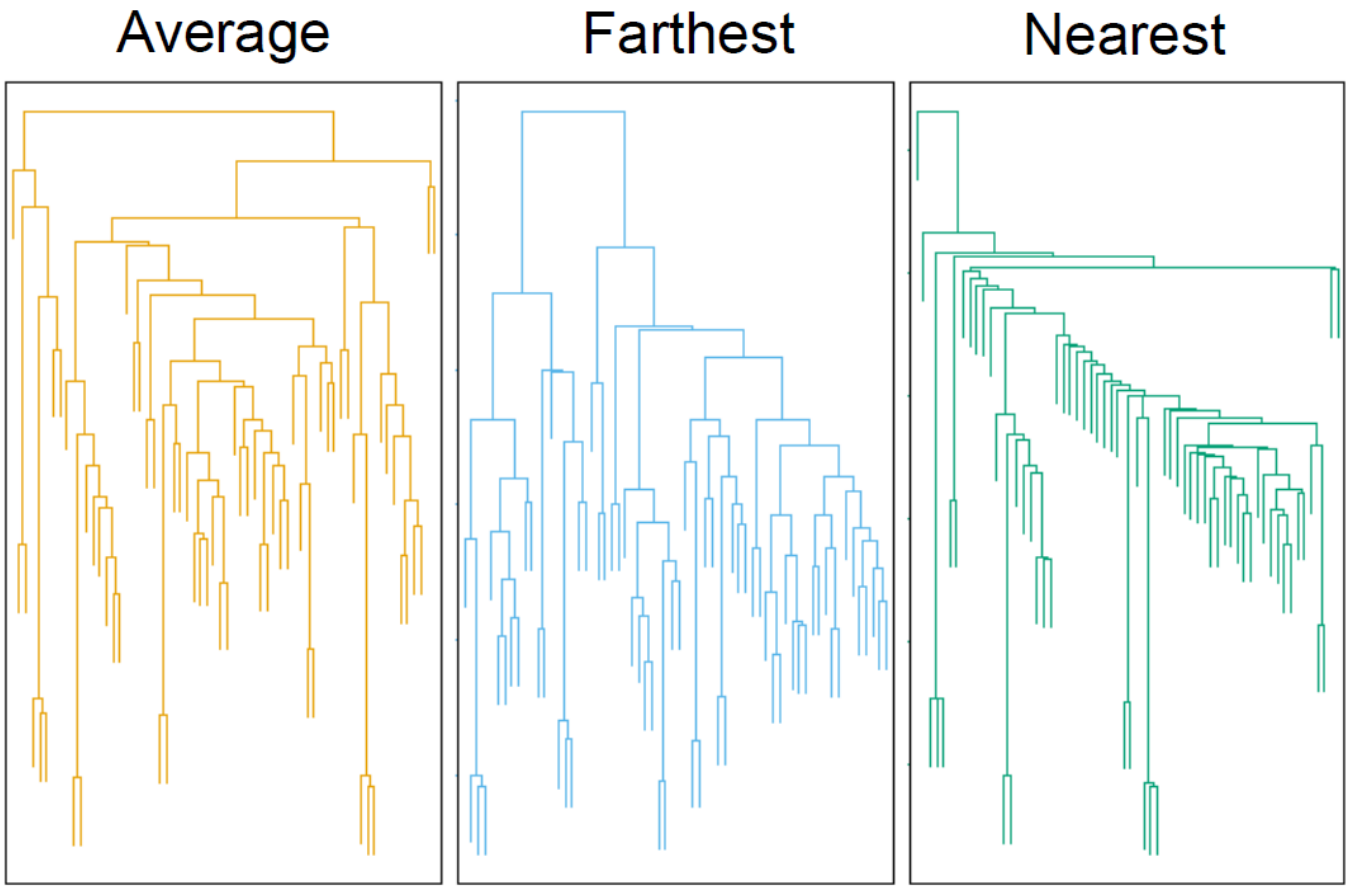
→

	c,d
a,b	6



簇与簇之间的距离采用最远距离

距离决定聚类



- **例子3：按最小距离准则对如下6个样本进行分级聚类：**

$$\mathbf{x}_1 = (0, 3, 1, 2, 0)$$
$$\mathbf{x}_2 = (1, 3, 0, 1, 0)$$
$$\mathbf{x}_3 = (3, 3, 0, 0, 1)$$
$$\mathbf{x}_4 = (1, 1, 0, 2, 0)$$
$$\mathbf{x}_5 = (3, 2, 1, 2, 1)$$
$$\mathbf{x}_6 = (4, 1, 1, 1, 0)$$

解：将每个样本单独看成一类，计算各点对之间的距离，见下表

	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$	$\mathbf{x}_6$
$\mathbf{x}_1$	0					
$\mathbf{x}_2$	$\sqrt{3}$	0				
$\mathbf{x}_3$	$\sqrt{15}$	$\sqrt{6}$	0			
$\mathbf{x}_4$	$\sqrt{6}$	$\sqrt{5}$	$\sqrt{13}$	0		
$\mathbf{x}_5$	$\sqrt{11}$	$\sqrt{8}$	$\sqrt{6}$	$\sqrt{7}$	0	
$\mathbf{x}_6$	$\sqrt{21}$	$\sqrt{14}$	$\sqrt{8}$	$\sqrt{11}$	$\sqrt{4}$	0

解 (续):

基于上述矩阵，根据最小距离准则，应将 $\mathbf{x}_1$和$\mathbf{x}_2$合并为一类，得到 $G_1=\{\mathbf{x}_1,\mathbf{x}_2\}$，$\{\mathbf{x}_3\}$，$\{\mathbf{x}_4\}$，$\{\mathbf{x}_5\}$，$\{\mathbf{x}_6\}$。**按最小距离原则**重新计算各类之间的距离，见下表：

	G_1	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$	$\mathbf{x}_6$
G_1	0				
$\mathbf{x}_3$	$\sqrt{6}$	0			
$\mathbf{x}_4$	$\sqrt{5}$	$\sqrt{13}$	0		
$\mathbf{x}_5$	$\sqrt{8}$	$\sqrt{6}$	$\sqrt{7}$	0	
$\mathbf{x}_6$	$\sqrt{14}$	$\sqrt{8}$	$\sqrt{11}$	$\sqrt{4}$	0

解释：比如，类G_1与$\{\mathbf{x}_3\}$之间的距离，按最小距离准则，计算为$\mathbf{x}_2$与$\mathbf{x}_3$的距离

解 (续)：

基于上述矩阵，根据最小距离准则，应将 $\mathbf{x}_5$和$\mathbf{x}_6$合并为一类，得到 $G_1=\{\mathbf{x}_1,\mathbf{x}_2\}$，$\{\mathbf{x}_3\}$，$\{\mathbf{x}_4\}$，$G_2=\{\mathbf{x}_5,\mathbf{x}_6\}$。**按最小距离原则**重新计算各类之间的距离，见下表：

	G_1	$\mathbf{x}_3$	$\mathbf{x}_4$	G_2
G_1	0			
$\mathbf{x}_3$	$\sqrt{6}$	0		
$\mathbf{x}_4$	$\sqrt{5}$	$\sqrt{13}$	0	
G_2	$\sqrt{8}$	$\sqrt{6}$	$\sqrt{7}$	0

解释：比如，类G_1与类G_2之间的距离，按最小距离准则，计算为$\mathbf{x}_2$与$\mathbf{x}_5$的距离。

解 (续)：

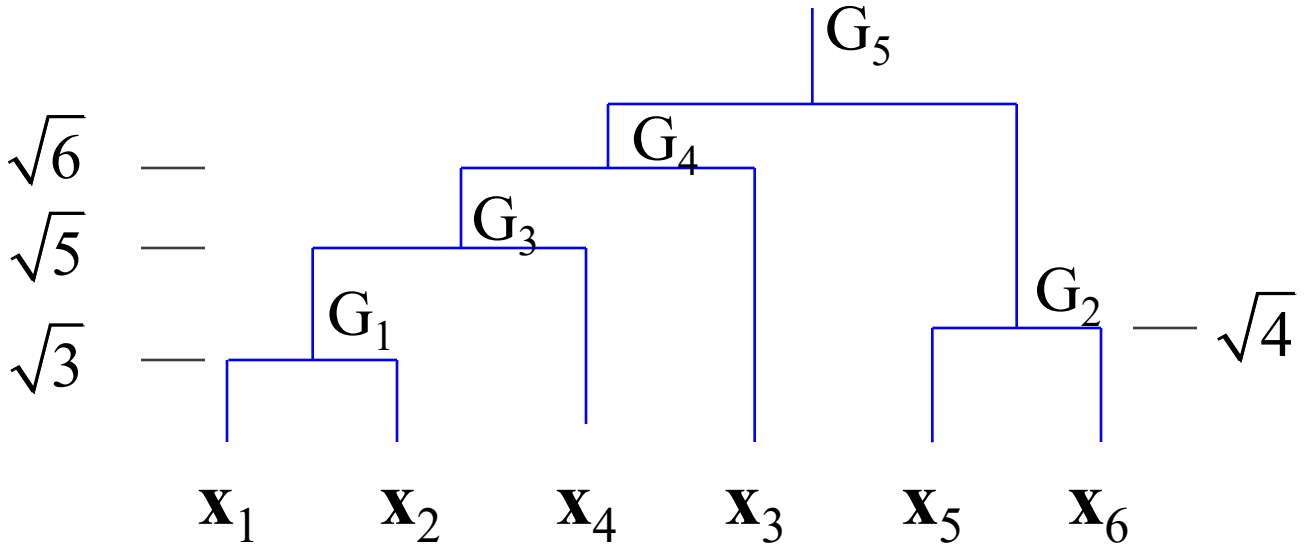
基于上述矩阵，根据最小距离准则，应将 G_1和$\mathbf{x}_3$合并为一类，得到 $G_3=G_1 \cup \{\mathbf{x}_4\}$，$\{\mathbf{x}_3\}$，$G_2=\{\mathbf{x}_5,\mathbf{x}_6\}$。**按最小距离原则**重新计算各类之间的距离，见下表：

	G_3	$\mathbf{x}_3$	G_2
G_3	0		
$\mathbf{x}_3$	$\sqrt{6}$	0	
G_2	$\sqrt{7}$	$\sqrt{6}$	0

解释：比如，类G_3与$\{\mathbf{x}_3\}$之间的距离，按最小距离准则，计算为$\mathbf{x}_2$与$\mathbf{x}_3$的距离

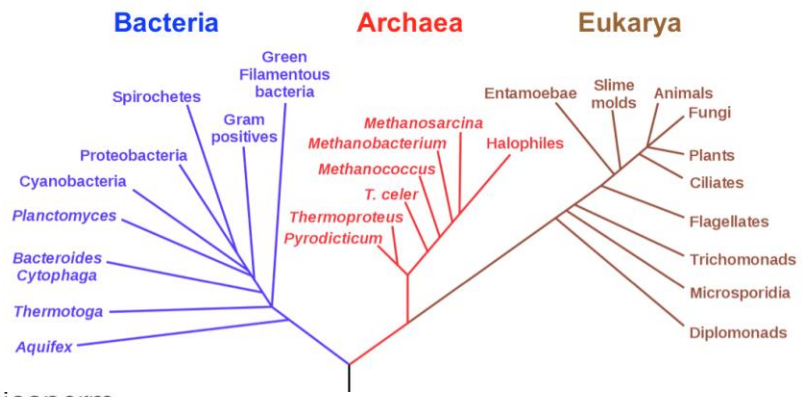
解 (续):

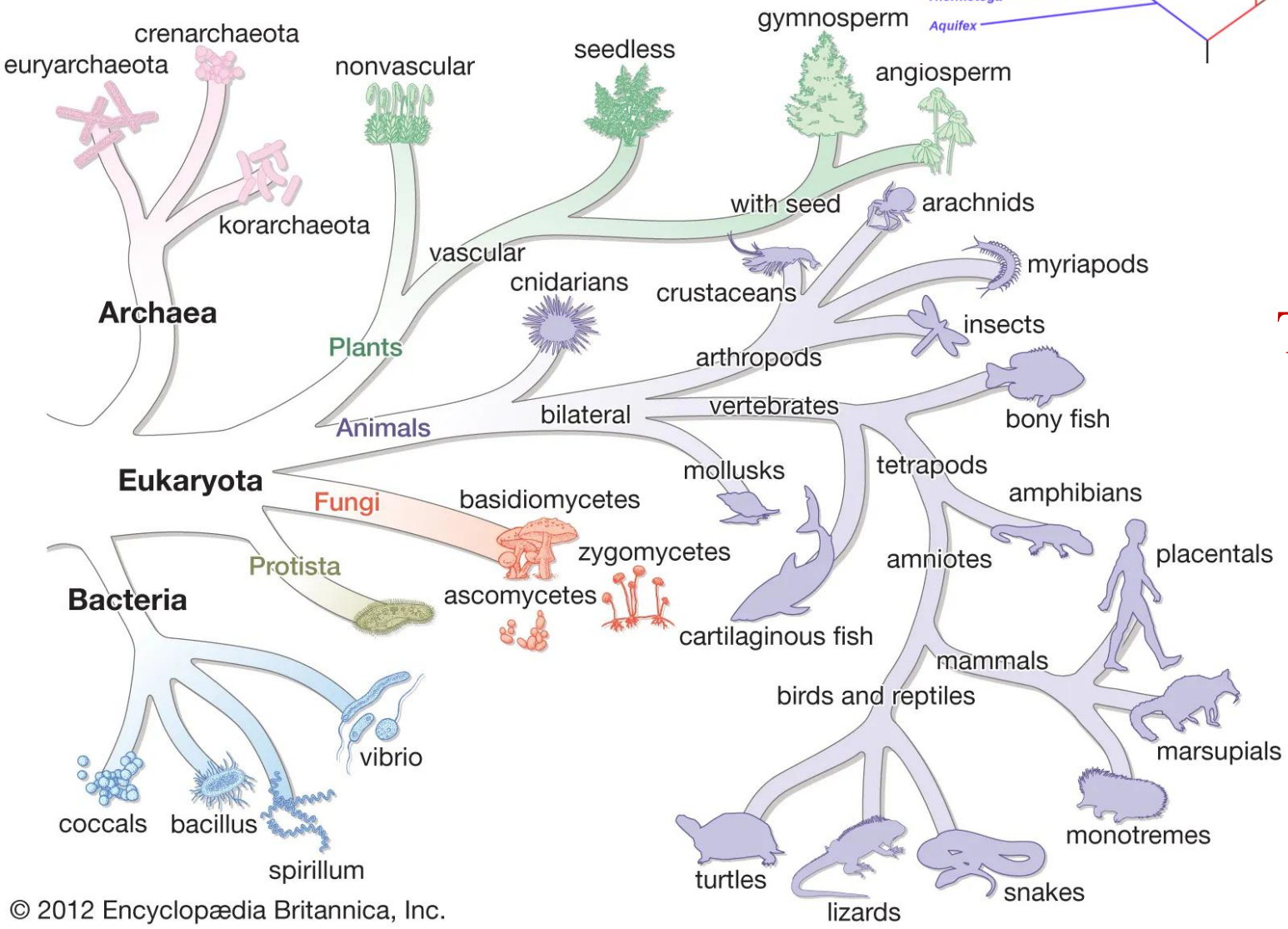
基于上述矩阵，根据最小距离准则，应将 G_3和$\mathbf{x}_3$合并为一类，当然也可以将 G_2 和 $\mathbf{x}_3$合并为一类。如选择前者，得到 $G_4=G_3 \cup \{\mathbf{x}_3\}$，$G_2=\{\mathbf{x}_5,\mathbf{x}_6\}$。最后，$G_4$和 G_2 合并为一类，这样所有数据将被合并在一起。



系统树图

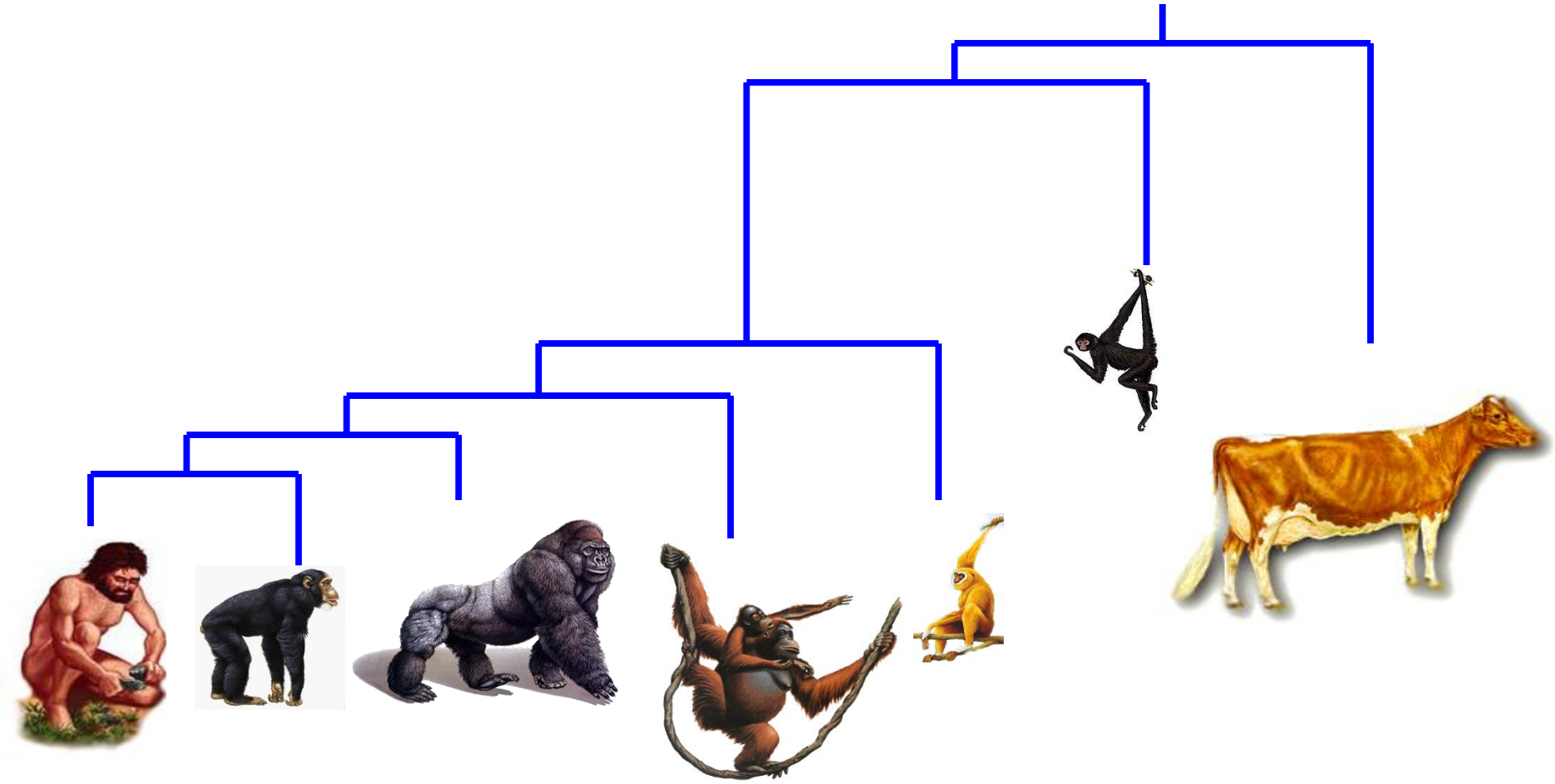
分级聚类应用



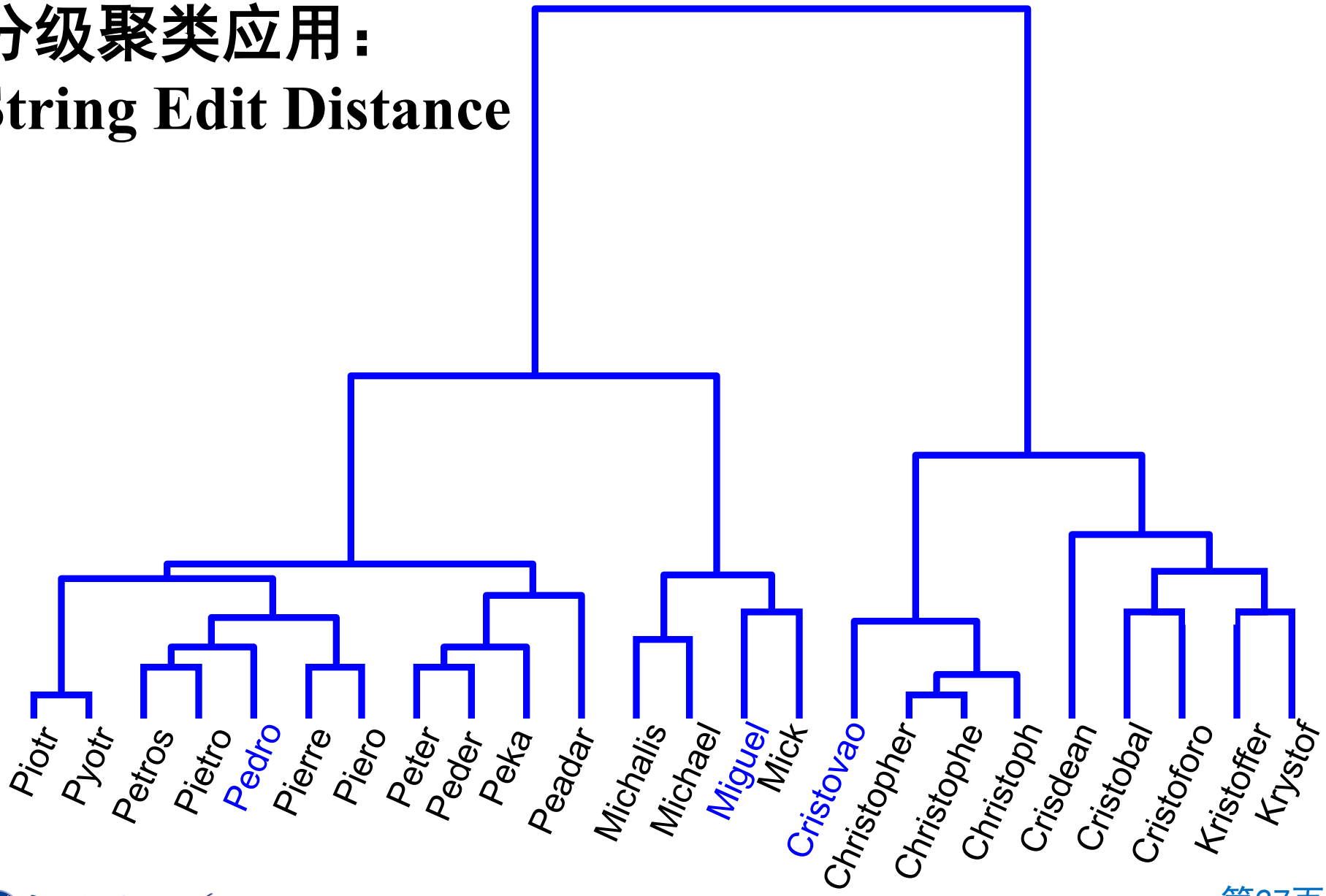


Tree of life

分级聚类应用



分级聚类应用：
String Edit Distance

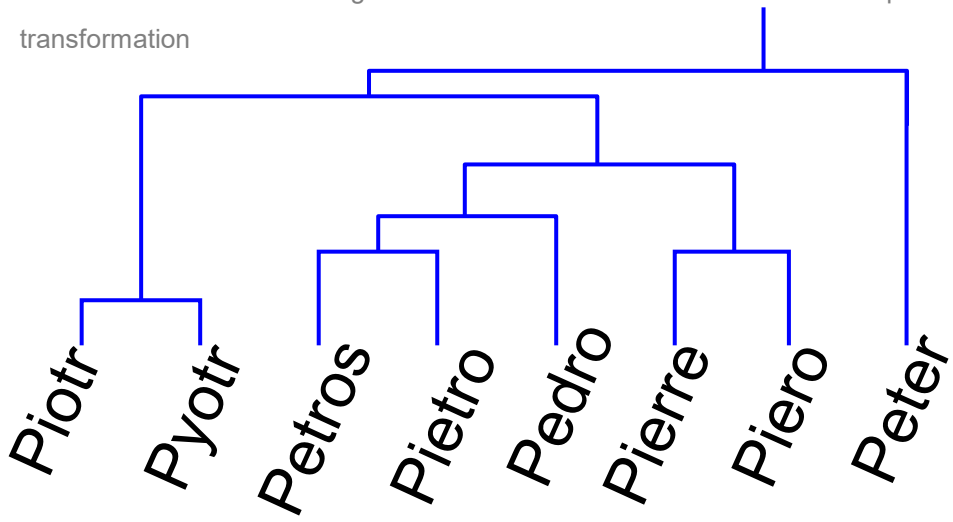


Edit Distance Example

It is possible to transform any string *Q* into string *C*, using only *Substitution*, *Insertion* and *Deletion*.
Assume that each of these operators has a cost associated with it.

The edit distance between two strings can be defined as the cost of the cheapest transformation from *Q* to *C*.

Note that for now we have ignored the issue of how we can find this cheapest transformation

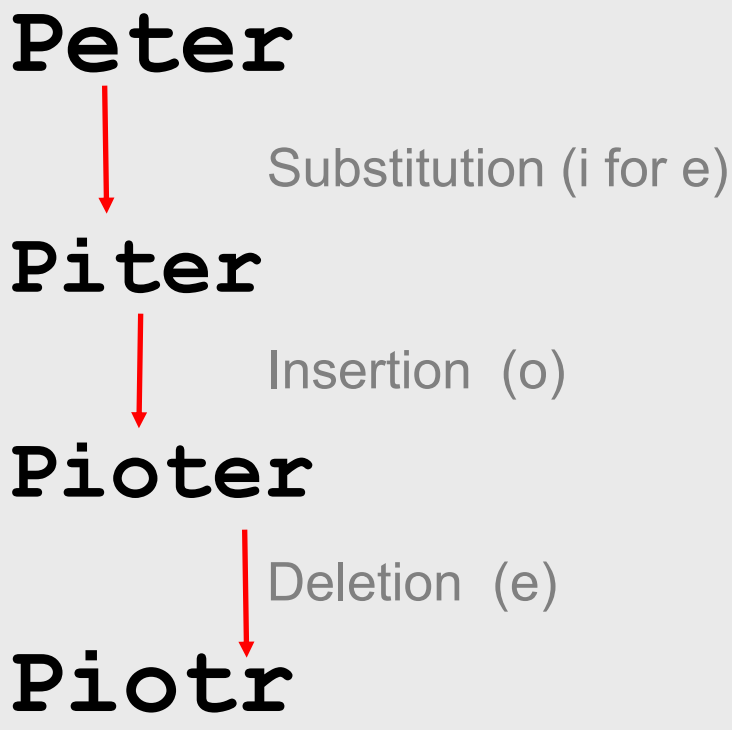


How similar are the names "Peter" and "Piotr"?

Assume the following cost function

Substitution	1 Unit
Insertion	1 Unit
Deletion	1 Unit

D(`Peter,Piotr`) is 3



String Edit Distance

To measure the similarity between two objects, transform one into the other, and measure how much effort it took. The measure of effort becomes the distance measure.

The distance between Patty and Selma:

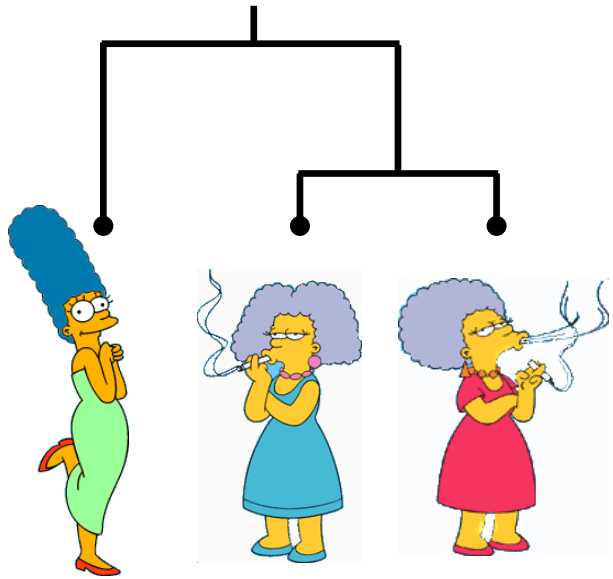
```
Change dress color,   1 point
Change earring shape, 1 point
Change hair part,     1 point
```

D(Patty, Selma) = **3**

The distance between Marge and Selma:

```
Change dress color,   1 point
Add earrings,         1 point
Decrease height,      1 point
Take up smoking,      1 point
Lose weight,          1 point
```

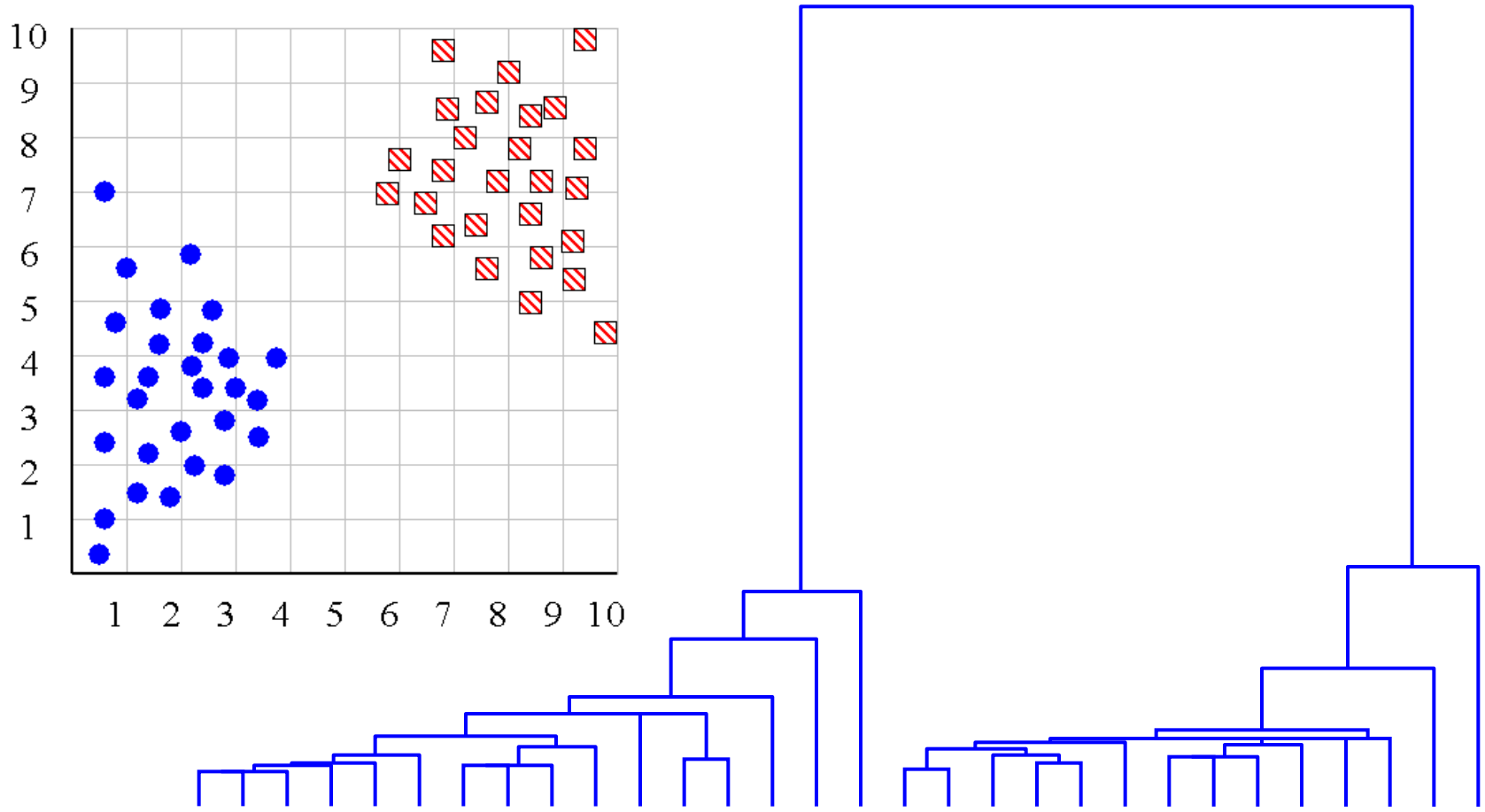
D(Marge, Selma) = **5**



Marge Patty Selma

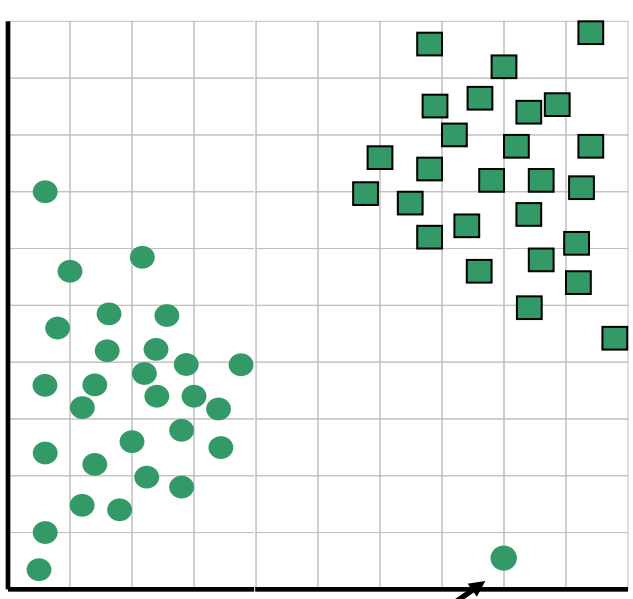
This is called the “edit distance” or the “transformation distance”

分级聚类用于选择类别数



分级聚类用于发现Outlier

The single isolated branch is suggestive of a data point that is very different to all others



Outlier

Hierarchical Clustering Methods Summary

- No need to specify the number of clusters in advance
- Hierarchical nature maps nicely onto human intuition for some domains
- They do not scale well: time complexity of at least $O(n^2)$, where n is the number of total objects
- Like any heuristic search algorithms, local optima are a problem
- Interpretation of results is subjective

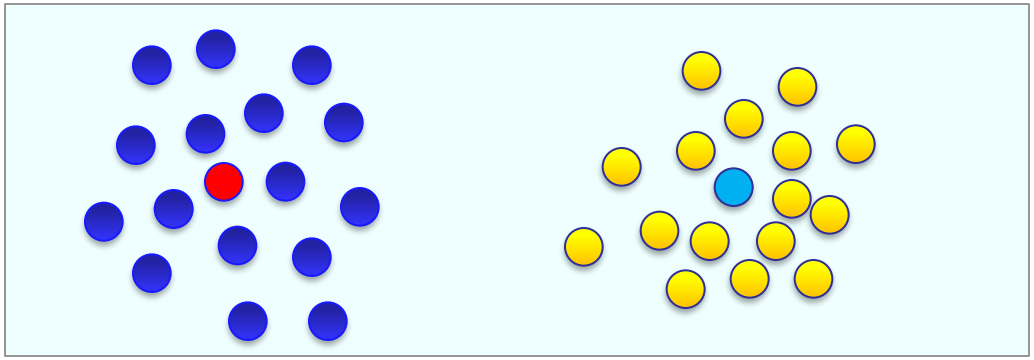
Spectral Clustering

谱聚类

→ *K*-means in spectrum space

引言

- **回忆K-means聚类**

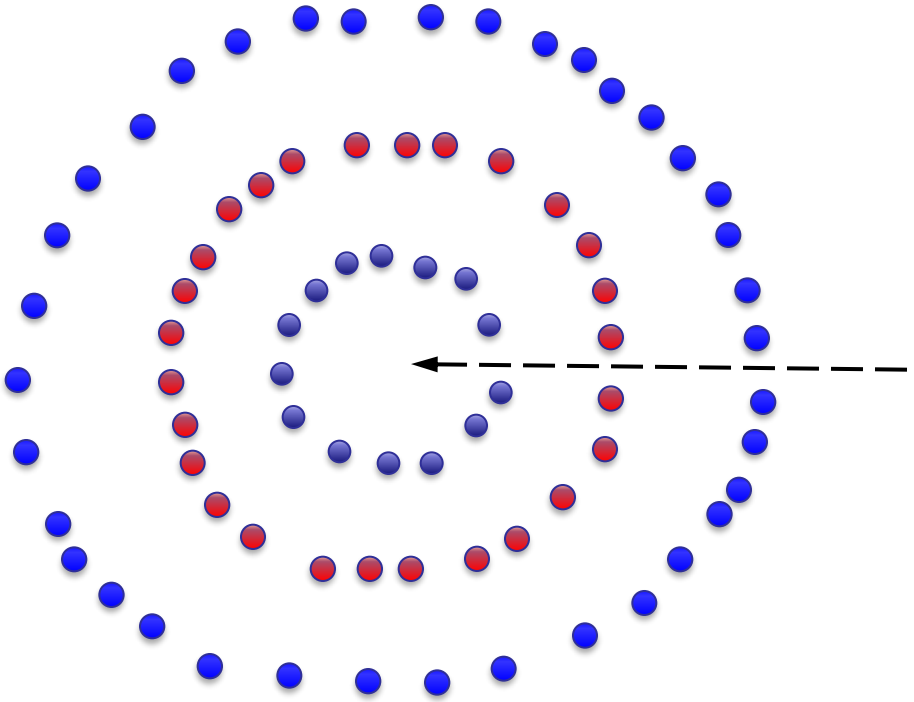


每个簇均可以用中心点来表示

(特别适合于单个簇符合高斯分布的情形)

引言

- 对于其它分布情形



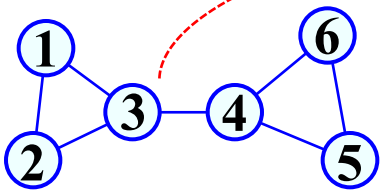
Spectral clustering allows us to address these sorts of clusters!

引言

- **谱学习方法**
  - 广义上讲，任何在学习过程中应用到矩阵特征值分解的方法均叫**谱学习方法**，比如主成分分析(PCA)、线性判别分析(LDA)、流形学习中的谱嵌入方法、谱聚类等等。
- **谱聚类**
  - 谱聚类算法建立在**图论**基础上，**其本质是将聚类问题转化为一个图上的顶点划分问题**。
  - 谱聚类算法建立在点对相似性基础之上，理论上能对任意分布形状的样本空间进行聚类。
  - 最早关于谱聚类的研究始于1973年，主要用于计算机视觉和VLSI设计领域。从2000年开始，谱聚类逐渐成为机器学习领域中的研究热点。

图论基本概念

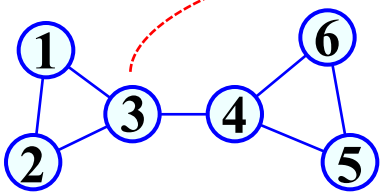
- **图 G：**由顶点集 V 和边集 E 所构成，记为 $G(V, E)$。根据边是否有向，可以分为无向图或者有向图。
- **邻接矩阵 W：**
  - 行数和列数等于矩阵顶点的个数；
  - $W_{ij} \in \{0,1\}$：1表示顶点i和j之间有边相连，0表示没有边相连。



$$\mathbf{W} = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}_{6\times 6}$$

图论基本概念

- **顶点的度：**等于与顶点相连接的边的条数。
- **度矩阵D：** 一个对角矩阵，$\mathbf{D}_{ii}$等于顶点i的度。

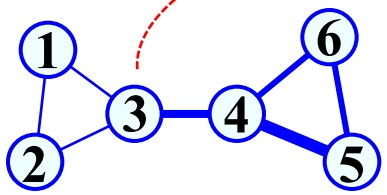


$$\mathbf{W}=\begin{pmatrix}0&1&1&0&0&0\\1&0&1&0&0&0\\1&1&0&1&0&0\\0&0&1&0&1&1\\0&0&0&1&0&1\\0&0&0&1&1&0\end{pmatrix}_{6\times6}$$

$$\mathbf{D}=\begin{pmatrix}2&0&0&0&0&0\\0&2&0&0&0&0\\0&0&3&0&0&0\\0&0&0&3&0&0\\0&0&0&0&2&0\\0&0&0&0&0&2\end{pmatrix}_{6\times6}$$

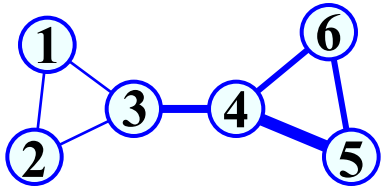
图论基本概念

- **有权图(weighted graph)**
  - W_{ij}为非负实数： $W_{ij}=0$表示顶点i与j之间没有边，W_{ij}越大表示顶点i,j越相似
  - 顶点的度： $d_i=\sum_{j\in V}W_{ij}$



$$\mathbf{W}=\begin{pmatrix} 0 & 0.1 & 0.1 & 0 & 0 & 0 \\ 0.1 & 0 & 0.2 & 0 & 0 & 0 \\ 0.1 & 0.2 & 0 & 1.5 & 0 & 0 \\ 0 & 0 & 1.5 & 0 & 2 & 1 \\ 0 & 0 & 0 & 2 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}_{6\times 6} \qquad \mathbf{D}=\begin{pmatrix} 0.2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1.8 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4.5 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 \end{pmatrix}_{6\times 6}$$

图论基本概念



- **拉普拉斯矩阵(Laplacian matrix)**
  - 度矩阵减去邻接矩阵得到拉普拉斯矩阵：**L= D - W**

$$\mathbf{W}=\begin{pmatrix} 0 & 0.1 & 0.1 & 0 & 0 & 0 \\ 0.1 & 0 & 0.2 & 0 & 0 & 0 \\ 0.1 & 0.2 & 0 & 1.5 & 0 & 0 \\ 0 & 0 & 1.5 & 0 & 2 & 1 \\ 0 & 0 & 0 & 2 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}_{6\times 6}$$

$$\mathbf{D}=\begin{pmatrix} 0.2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1.8 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4.5 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 \end{pmatrix}_{6\times 6}$$

$$\mathbf{L}=\mathbf{D}-\mathbf{W}=\begin{pmatrix} 0.2 & -0.1 & -0.1 & 0 & 0 & 0 \\ -0.1 & 0.3 & -0.2 & 0 & 0 & 0 \\ -0.1 & -0.2 & 1.8 & -1.5 & 0 & 0 \\ 0 & 0 & -1.5 & 4.5 & -2 & -1 \\ 0 & 0 & 0 & -2 & 3 & -1 \\ 0 & 0 & 0 & -1 & -1 & 2 \end{pmatrix}_{6\times 6}$$

图论基本概念

- 拉普拉斯矩阵(Laplacian matrix)
  - 对称归一化

$$\mathbf{L}_{sym} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{W}\mathbf{D}^{-1/2}$$

  - 非对称归一化

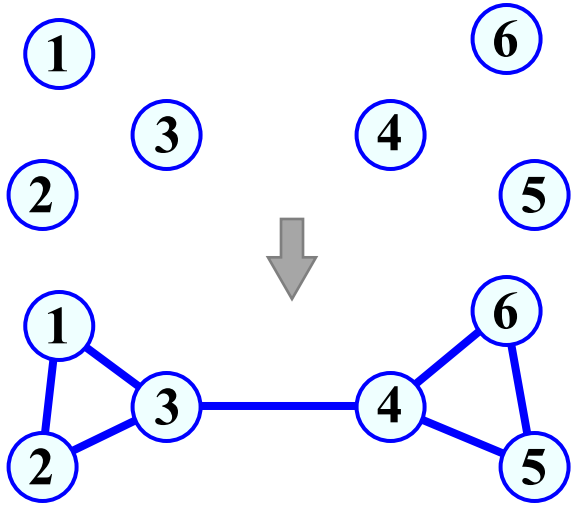
$$\mathbf{L}_{rw} = \mathbf{D}^{-1}\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{W}$$

图论基本概念

- **基于数据集的图构造**

(1) 构造顶点集V: 对每一个样本构造一个顶点

$$\{\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_6\} \subset R^d$$



(2) 构造边集E

局部连接

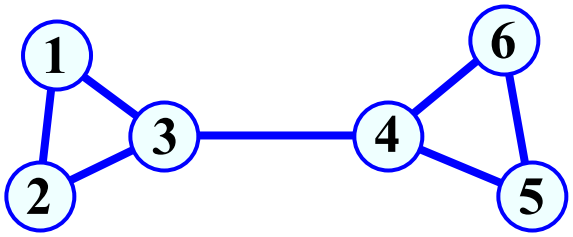
- ***k* - 近邻**：对每个样本$\mathbf{x}_i$，在所有样本中找出它的 k 个最邻近，在 $\mathbf{x}_i$ 与每个邻近样本间添加一条边
- **ε - 近邻**：对每个样本$\mathbf{x}_i$，在所有样本中找出与它距离小于的 ε 的样本，在 $\mathbf{x}_i$ 与这些样本间添加一条边

图论基本概念

- **基于数据集的图构造**

(3) 构造邻接矩阵**W**：对任意两个节点，若它们相连，则根据某种相似度计算其权重；若不相连，则权重为零

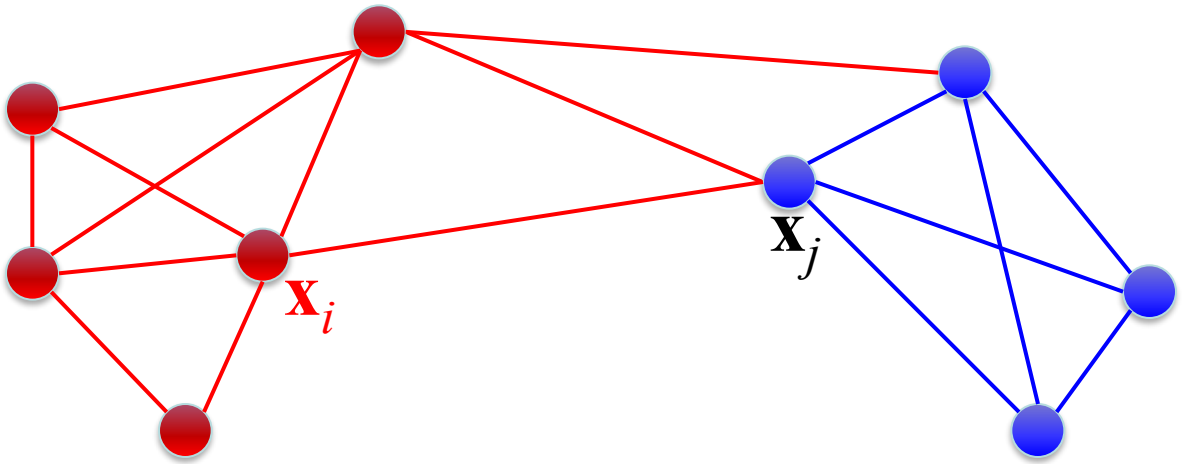
$$w_{ij}=\begin{cases}\exp(-\dfrac{\|\mathbf{x}_i-\mathbf{x}_j\|^2}{2\sigma^2}), & \text{if } <i,j>\in E\\ 0, & \text{otherwise}\end{cases}$$



$$\mathbf{W}=\begin{pmatrix}0 & w_{12} & w_{13} & 0 & 0 & 0\\ & 0 & w_{23} & 0 & 0 & 0\\ & & 0 & w_{34} & 0 & 0\\ & \text{对} & & 0 & w_{45} & w_{46}\\ & & \text{称} & & 0 & w_{56}\\ & & & & & 0\end{pmatrix}$$

图论基本概念

- 子图 A⊂V的势 |A|：等于A所包含的顶点个数
- 子图 A⊂V的体积 vol(A)：等于A中所有顶点的度之和



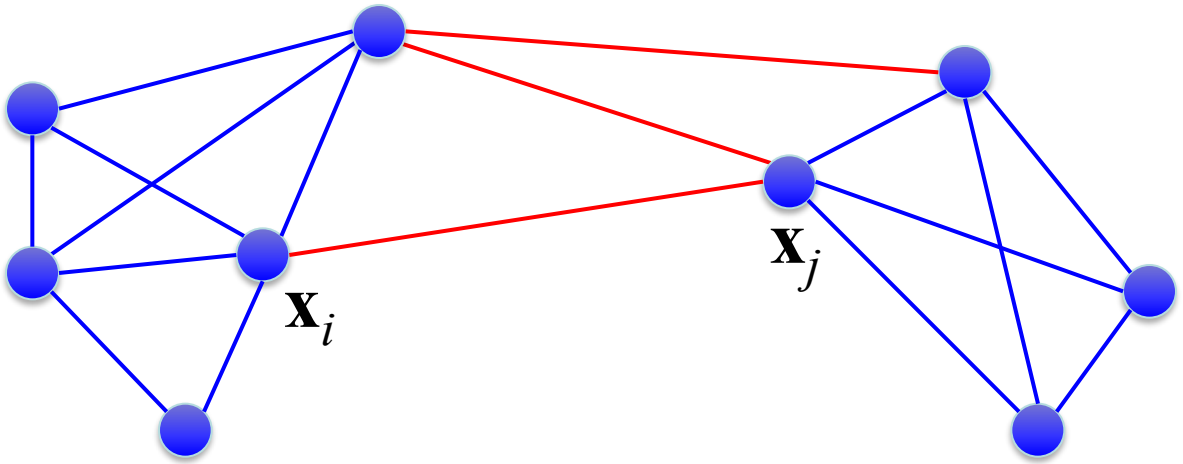
$$vol(A) = \sum_{i \in A} d_i$$

图论基本概念

- **子图A的补图：** V 中去掉 A 中顶点所构成的子图 $\bar{A}$

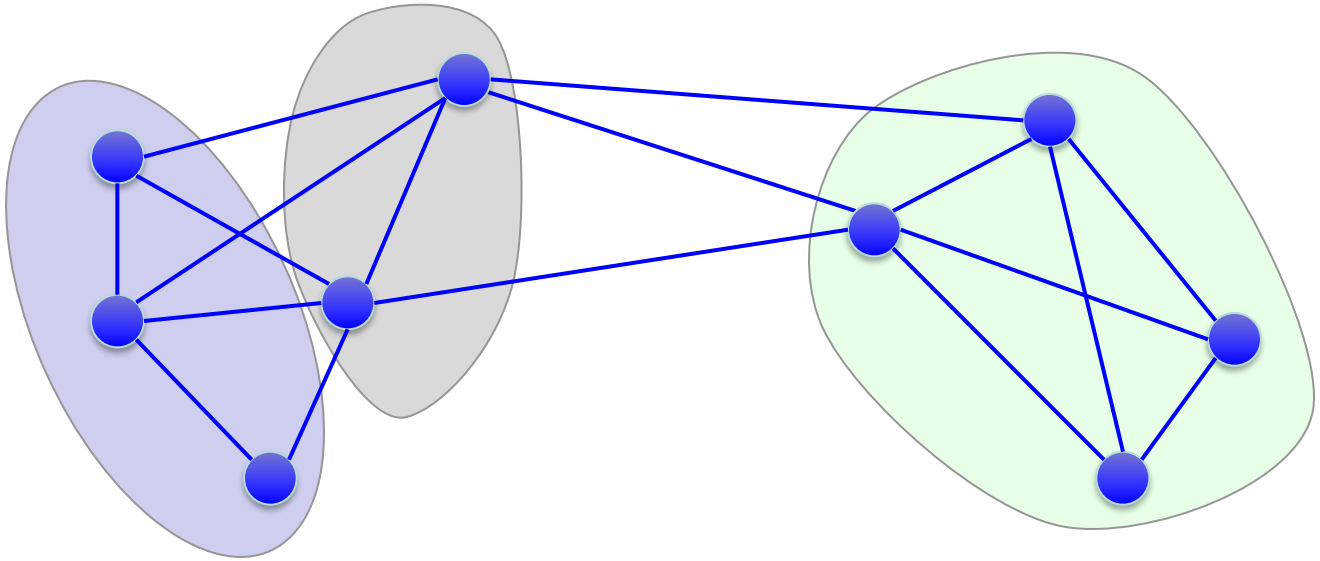
$$\bar{A} = V - A$$

- **边割：** 边 E 的一个子集，去掉该子集中的边，图就变成两个不连通的子图。



图论基本概念

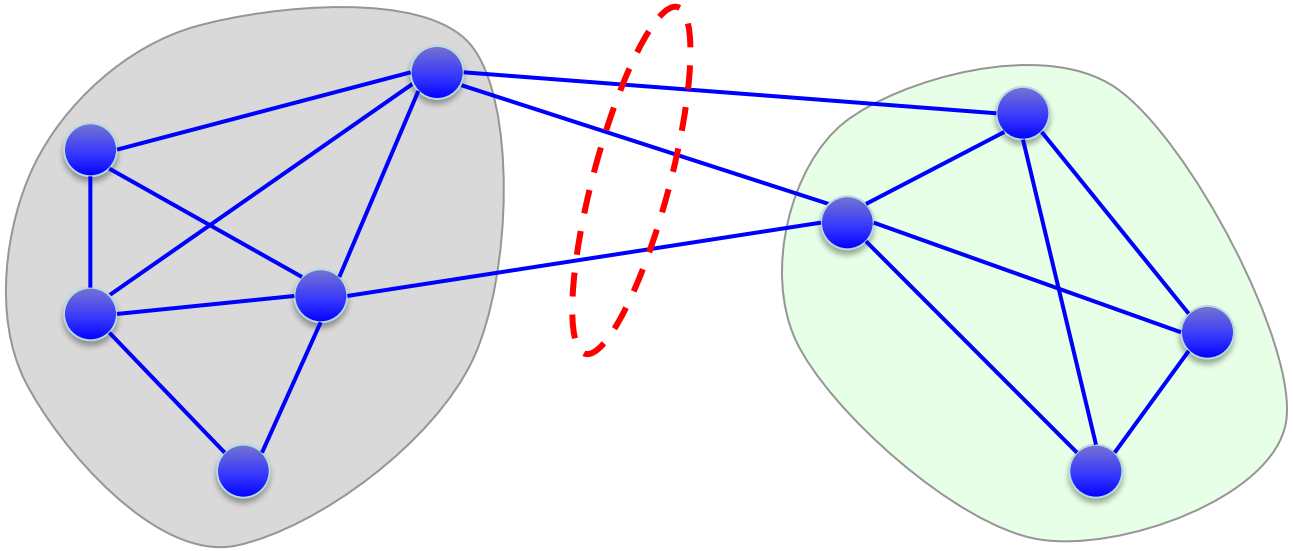
- **图切割：**
  - 设 $A_1, A_2, \ldots, A_K$ 为顶点集合 V 的非空连通子集，如果 $A_i \cap A_j = \varnothing, i \neq j$，且 $A_1 \cup A_2 \cup \ldots \cup A_K = V$, 则称 $A_1, A_2, \ldots, A_K$ 为图 G 的一个切割。



图论基本概念

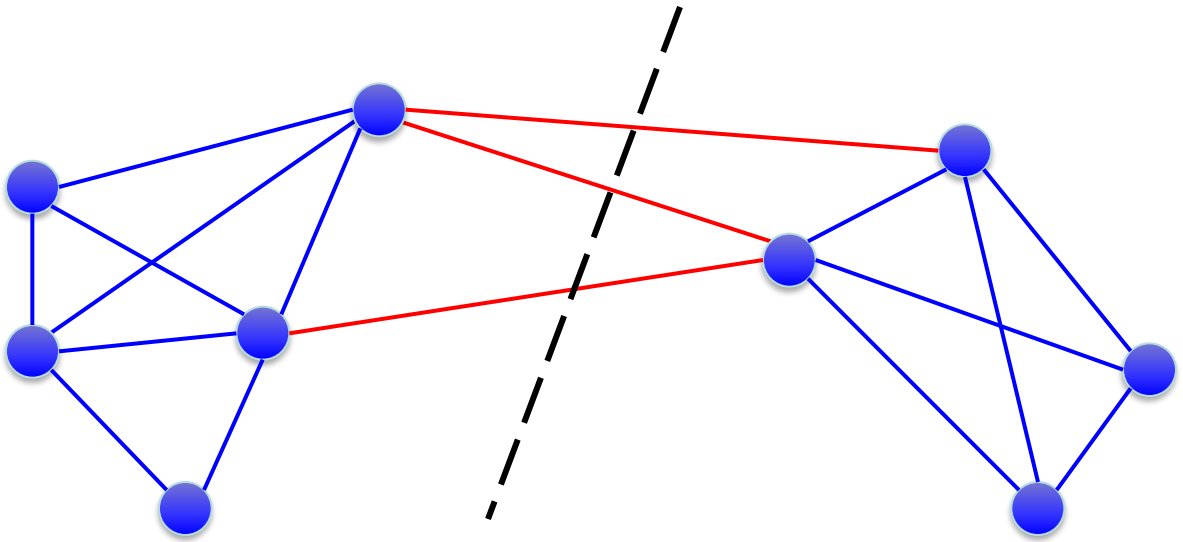
- **子图 A 与子图 B 的相似度/子图 A 与子图 B 的切割：**定义为连接两个子图所有边的权重之和

$$\mathrm{cut}(A,B)=W(A,B)=\sum_{i\in A,j\in B} w_{ij}$$



图切割

- **最小二分切割 (Minimum bipartitional cut)**
  - 在所有的图切割中，找一个最小代价的切割，将图分为两个不连通的子图。也就是说，被切开的两个子图之间的相似性最小。



图切割

- **最小二分切割**
  - 在所有的图切割中，找一个最小代价的切割，将图分为两个不连通的子图。也就是说，被切开的两个子图之间的相似性最小。最优化问题如下：

$$\min_{A \subset V} \quad \mathrm{cut}(A, \bar{A}) := W(A, \bar{A}) = \sum_{i \in A, j \in \bar{A}} w_{ij}$$

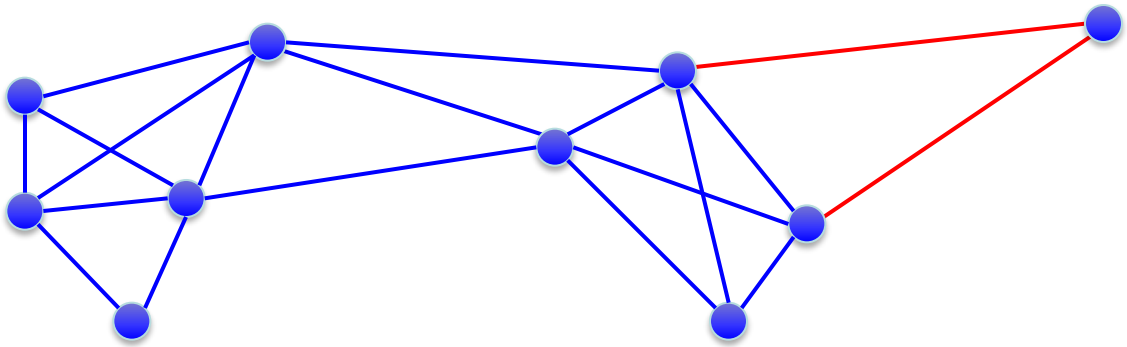
$$\text{s.t.} \quad A \neq \varnothing,$$

$$A \cap \bar{A} = \varnothing,$$

$$A \cup \bar{A} = V$$

图切割

- 最小二分切割
  - **在实践中，上述目标函数通常将一个点(比如野点)从其余各点中分离出来**。从聚类的角度看，这并不是我们所期望的。



问题的原因在于没有对子图的规模加以限制

图切割

- 归一化最小二分切割
  - 一个基本的假设是希望两个子图的**规模不要相差太大。**
  - 基本做法是用**子图的势或者体积**对切割进行归一化，即定义如下目标函数：
    - 采用子图的势：

$$\mathrm{Ratiocut}(A,\bar{A}) := \frac{1}{2}\left(\frac{cut(A,\bar{A})}{|A|}+\frac{cut(A,\bar{A})}{|\bar{A}|}\right)$$

    - 采用子图的体积：

$$\mathrm{Ncut}(A,\bar{A}) := \frac{1}{2}\left(\frac{cut(A,\bar{A})}{vol(A)}+\frac{cut(A,\bar{A})}{vol(\bar{A})}\right)$$

图切割

- **K-切割 ($K > 2$)：**
  - 考虑将图分成 K 个子图：$A_1, A_2, \ldots, A_K$。一种直接的方法是将K-切割看作多个二分切割问题。
- **未归一化切割目标函数：**

$$\mathrm{cut}(A_1, A_2, \cdots, A_K) := \sum_{i=1}^{K} W(A_i, \bar{A}_i)$$

- **比例切割目标函数：**

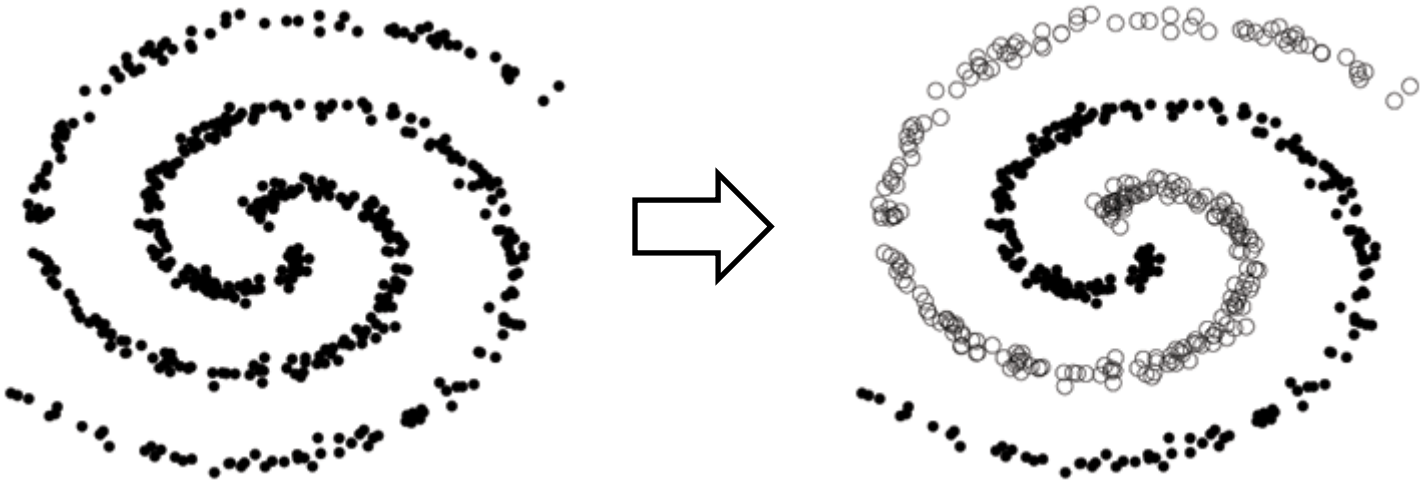
$$\mathrm{Ratiocut}(A_1, A_2, \cdots, A_K) := \sum_{i=1}^{K} \frac{W(A_i, \bar{A}_i)}{|A_i|} = \sum_{i=1}^{K} \frac{\mathrm{cut}(A_i, \bar{A}_i)}{|A_i|}$$

- **归一化切割目标函数：**

$$\mathrm{Ncut}(A_1, A_2, \cdots, A_K) := \sum_{i=1}^{K} \frac{W(A_i, \bar{A}_i)}{\mathrm{vol}(A_i)} = \sum_{i=1}^{K} \frac{\mathrm{cut}(A_i, \bar{A}_i)}{\mathrm{vol}(A_i)}$$

Spectral Clustering

- Spectral Clustering
  - Simple, but powerful method of clustering
  - Requires less assumptions on the form of clusters
  - Outperforms the traditional approaches, such as *k*-means clustering.



Spectral Clustering

- Spectral Method
  - Spectral analysis in linear algebra
    - Basic features of **matrices**: eigenpairs (eigenvalue, eigenvector)
    - Methods of using the eigenpairs to solve given problems

- Spectral Clustering
  - Methods of **using the eigenvectors of some matrices** to find a partition of the data such that points in the same group are similar

Spectral Clustering Algorithm 1

- Unnormalized Spectral Clustering

1. Construct a similarity graph and compute the unnormalized graph Laplacian L.
2. Compute the k smallest eigenvectors $u_1, u_2, \cdots, u_k$ of L.
3. Let $U = [\, u_1 \; u_2 \; \cdots \; u_k \,] \in \mathbb{R}^{n \times k}$.
4. Let $y_i \in \mathbb{R}^k$ be the vector corresponding to the ith row of U.

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1k} \\ u_{21} & u_{22} & \cdots & u_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ u_{n1} & u_{n2} & \cdots & u_{nk} \end{bmatrix} = \begin{bmatrix} y_1^T \\ y_2^T \\ \vdots \\ y_n^T \end{bmatrix}$$

n: number of data points

5. Thinking of y_i's as points in $\mathbb{R}^k$, cluster them with k-means algorithms.

Spectral Clustering Algorithm 2

- Normalized Spectral Clustering [Shi2000]

1. Construct a similarity graph and compute the normalized graph Laplacian L_{rw}
2. Compute the k smallest eigenvectors $u_1, u_2, \cdots, u_k$ of L_{rw}
3. Let $U = [\, u_1\ u_2\ \cdots\ u_k\,] \in \mathbb{R}^{n\times k}$.
4. Let $y_i \in \mathbb{R}^k$ be the vector corresponding to the ith row of U.

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1k} \\ u_{21} & u_{22} & \cdots & u_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ u_{n1} & u_{n2} & \cdots & u_{nk} \end{bmatrix} = \begin{bmatrix} y_1^T \\ y_2^T \\ \vdots \\ y_n^T \end{bmatrix}$$

5. Thinking of y_i's as points in $\mathbb{R}^k$, cluster them with k-means algorithms.

Spectral Clustering Algorithm 3

- Normalized Spectral Clustering [Ng2002]

1. Construct a similarity graph and compute the normalized graph Laplacian L_{sym}.
2. Compute the k smallest eigenvectors $u_1, u_2, \cdots, u_k$ of L_{sym}.
3. Let $U = [\, u_1\ u_2\ \cdots\ u_k\,] \in \mathbb{R}^{n\times k}$.
4. Normalized the rows of U to norm 1.

$$U_{ij} \leftarrow \frac{U_{ij}}{(\sum_k U_{ik}^2)^{1/2}}$$

5. Let $y_i \in \mathbb{R}^k$ be the vector corresponding to the ith row of U.
6. Thinking of y_i's as points in $\mathbb{R}^k$, cluster them with k-means algorithms.

Spectral Clustering

Simulated example: Graph of 3 connected components

$$W=\begin{bmatrix}1&1&1&0&0&0&0&0\\1&1&1&0&0&0&0&0\\1&1&1&0&0&0&0&0\\0&0&0&1&1&1&0&0\\0&0&0&1&1&1&0&0\\0&0&0&1&1&1&0&0\\0&0&0&0&0&0&1&1\\0&0&0&0&0&0&1&1\end{bmatrix}$$

Eigenvalues

[3.0 3.0 3.0 3.0 2.0 0.0 0.0 -0.0]

Eigenvectors

$$\begin{bmatrix}0.0&0.0&0.0&0.8&-0.0&0.0&-0.6&0.0\\0.0&0.0&0.7&-0.4&-0.0&0.0&-0.6&0.0\\0.0&0.0&-0.7&-0.4&-0.0&0.0&-0.6&0.0\\0.0&0.8&0.0&0.0&-0.0&-0.6&0.0&0.0\\0.7&-0.4&0.0&0.0&-0.0&-0.6&0.0&0.0\\-0.7&-0.4&0.0&0.0&-0.0&-0.6&0.0&0.0\\0.0&0.0&0.0&0.0&0.7&0.0&0.0&0.7\\0.0&0.0&0.0&0.0&-0.7&0.0&0.0&0.7\end{bmatrix}$$

$$W=\begin{bmatrix}1&1&0&0&0&1&0&0\\1&1&0&0&0&1&0&0\\0&0&1&0&1&0&0&1\\0&0&0&1&0&0&1&0\\0&0&1&0&1&0&0&1\\1&1&0&0&0&1&0&0\\0&0&0&1&0&0&1&0\\0&0&1&0&1&0&0&1\end{bmatrix}$$

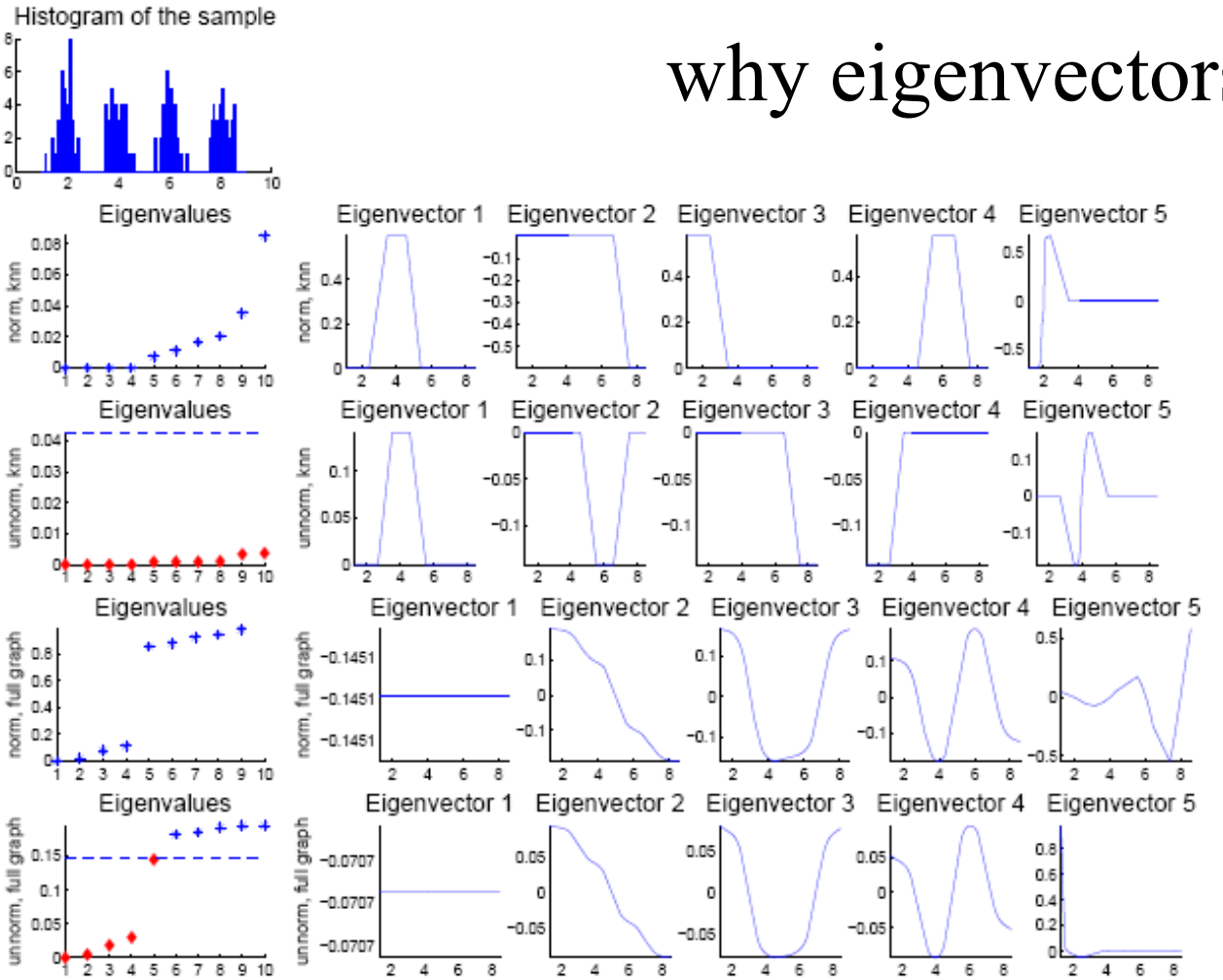
Eigenvalues

[3.0 3.0 3.0 3.0 2.0 0.0 0.0 0.0]

Eigenvectors

$$\begin{bmatrix}0.8&0.0&0.0&0.0&0.0&0.0&-0.0&-0.6\\-0.4&0.0&-0.0&-0.7&0.0&-0.0&-0.0&-0.6\\0.0&-0.0&0.8&0.0&-0.0&0.0&0.6&0.0\\0.0&-0.0&-0.0&0.0&-0.7&0.7&0.0&0.0\\0.0&-0.7&-0.4&0.0&0.0&0.0&0.6&0.0\\-0.4&-0.0&0.0&0.7&-0.0&0.0&0.0&-0.6\\0.0&0.0&0.0&0.0&0.7&0.7&-0.0&0.0\\0.0&0.7&-0.4&0.0&-0.0&-0.0&0.6&0.0\end{bmatrix}$$

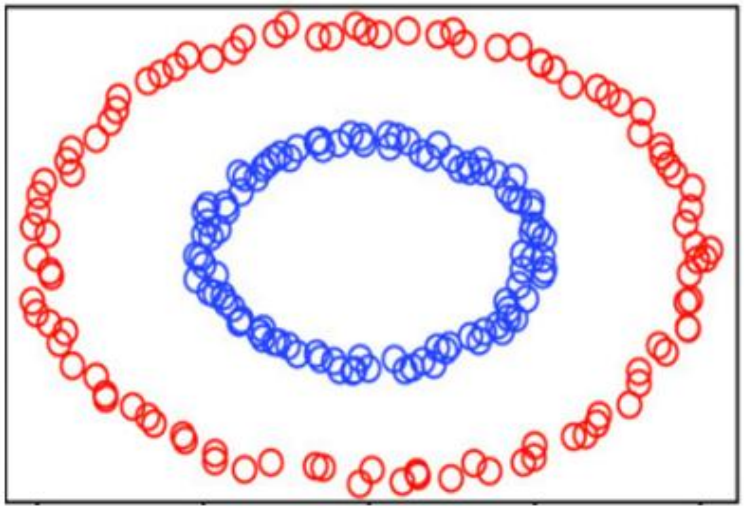
Spectral Clustering

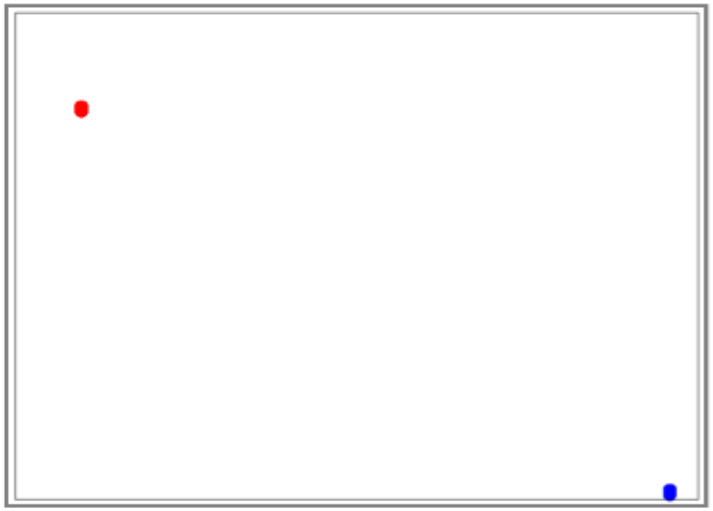


why eigenvectors?

Spectral Clustering

Represents the data using K eigenvectors and obtains the clusters





Two Explanations

- Graph partitioning point of view
  - Based on Mincut problem in a similarity graph
  - Find a partition such that the edges between clusters have a very low weight and the edges within a cluster have high weight.

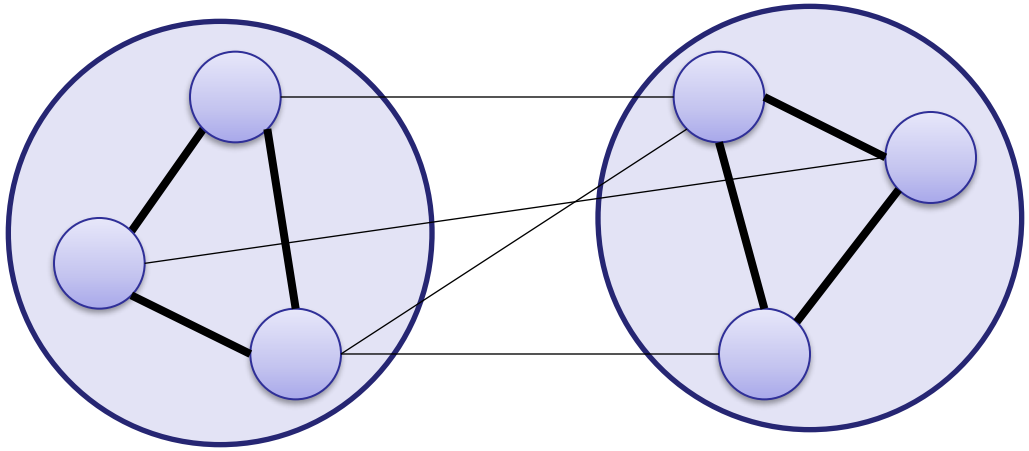
- Random walks point of view
  - Based on random walks on the similarity graph
  - Find a partition such that random walk stays long within the same cluster and seldom jumps to other clusters.

Graph Partitioning Point of View

- Mincut problem
  - Given a number K, find a partition $A_1, A_2, \ldots, A_K$ which minimizes

$$\text{cut}(A_1, A_2, \cdots, A_K) := \sum_{i=1}^{K} W(A_i, \overline{A_i})$$

$$W(A_i, \overline{A_i}) = \sum_{p \in A_i,\, q \in \overline{A_i}} W_{pq}$$



- In practice, the Mincut problem results in a size imbalance in the partition.

Graph Partitioning Point of View

- Two common objective functions

$$\text{Ratiocut}(A_1, A_2, \cdots, A_K) := \sum_{i=1}^{K} \frac{W(A_i, \overline{A_i})}{|A_i|} = \sum_{i=1}^{K} \frac{\text{cut}(A_i, \overline{A_i})}{|A_i|}$$

$$\text{Ncut}(A_1, A_2, \cdots, A_K) := \sum_{i=1}^{K} \frac{W(A_i, \overline{A_i})}{\text{vol}(A_i)} = \sum_{i=1}^{K} \frac{\text{cut}(A_i, \overline{A_i})}{\text{vol}(A_i)}$$

$$\text{vol}(A_i) = \sum_{k \in A_i} D_{kk}$$

- Normalized Mincut problem
  - Given a number K, find a partition $A_1, A_2, \ldots, A_K$ which minimizes Ratiocut or Ncut.

Graph Partitioning Point of View

- Indicator vector of a subset A

$$f_A[i] = \begin{cases} 1, & v_i \in A \\ 0, & \text{otherwise} \end{cases} \qquad f_A \in \{0,1\}^n$$

- Relationship between graph cut and graph Laplacian

$$\text{cut}(A, \overline{A}) = f_A^T L f_A$$

- Let $\{f_1, \ldots, f_k\} \subset \{0,1\}^n$ be k indicator vectors for $A_1, A_2, \ldots, A_k$

$$\text{cut}(A_1, A_2, \ldots, A_k) = \sum_{i=1}^{k} \text{cut}(A_i, \overline{A_i}) = \sum_{i=1}^{k} f_i^T L f_i = \text{tr}(F^T L F)$$

$$F = [f_1, f_2, \ldots, f_k] \in \{0,1\}^{n \times k}$$

- Mincut problem is converted into

$$\arg\min_{F \in R^{n \times k}} \text{tr}(F^T L F)$$

$$\text{s.t. } F_{ij} \in \{0,1\} \text{ and } F^T F = I \in R^{k \times k}$$

Graph Partitioning Point of View

Key property of L: $f_i^T L f_i = \text{cut}(A_i, \overline{A_i})$
for all $f \in R^n$

$$
\begin{aligned}
& f^T \mathbf{L} f \\
&= f^T \mathbf{D} f - f^T \mathbf{W} f \\
&= \sum_{i=1}^{n} d_i f_i^2 - \sum_{i,j=1}^{n} w_{ij} f_i f_j \\
&= \frac{1}{2}\left(\sum_{i=1}^{n}\left(\sum_{j=1}^{n} w_{ij} \right) f_i^2 - 2\sum_{i,j=1}^{n} w_{ij} f_i f_j + \sum_{j=1}^{n}\left(\sum_{i=1}^{n} w_{ij} \right) f_j^2 \right) \\
&= \frac{1}{2} \sum_{i,j=1}^{n} w_{ij} (f_i - f_j)^2
\end{aligned}
$$

Graph Partitioning Point of View

- Relaxing the form of indicator vectors, normalized mincut problems can be expressed with graph Laplacian.

$$f_{ij} = \begin{cases} 1/\sqrt{|A_j|} & \text{if } v_i \in A_j \\ 0 & \text{otherwise} \end{cases}$$

$$\text{RatioCut Problem} \Rightarrow \begin{array}{l} \arg\min_{F \in \mathbb{R}^{n \times k}} \operatorname{tr}(F^T L F) \\ \text{subject to } F^T F = I, \ f_i\text{'s are indicator vectors} \end{array}$$

$$f_{ij} = \begin{cases} 1/\sqrt{\operatorname{vol}(A_j)} & \text{if } v_i \in A_j \\ 0 & \text{otherwise} \end{cases}$$

$$\text{NCut Problem} \Rightarrow \begin{array}{l} \arg\min_{F \in \mathbb{R}^{n \times k}} \operatorname{tr}(F^T L F) \\ \text{subject to } F^T D F = I, \ f_i\text{'s are indicator vectors} \end{array}$$

Graph Partitioning Point of View

- The optimization problems are NP-hard.
  - Relax the discreteness condition of vectors $f_1, \cdots, f_k$.

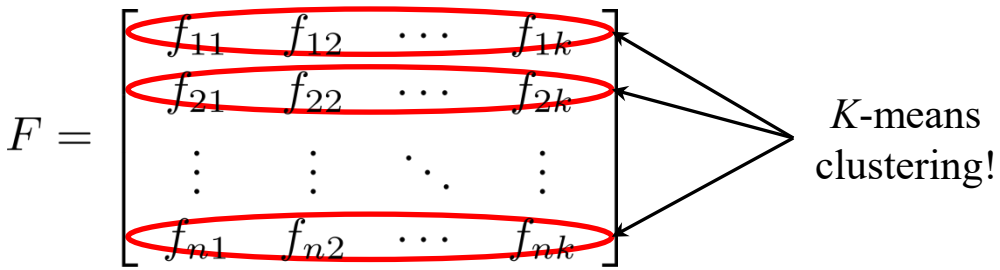
$$\text{RatioCut Problem} \Rightarrow \begin{array}{l} \arg\min \operatorname{tr}(F^T L F) \\ F \in \mathbb{R}^{n \times k} \\ \text{subject to } F^T F = I \end{array}$$

$$\text{NCut Problem} \Rightarrow \begin{array}{l} \arg\min \operatorname{tr}(F^T L F) \\ F \in \mathbb{R}^{n \times k} \\ \text{subject to } F^T D F = I \end{array}$$

- By the Rayleigh-Ritz theorem, the solutions of above problems are the matrix that contains the *k* smallest eigenvectors of the graph Laplacian L and normalized one L_{sym}, respectively.

Graph Partitioning Point of View

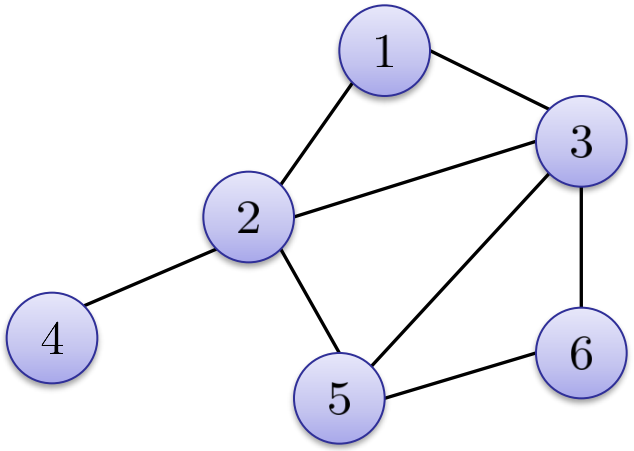
- Note that the solutions of the relaxed optimization problems does NOT indicate which nodes are included in which groups!
- However, we hope that if the data are *well-separate*, the eigenvectors of graph Laplacians are close to piecewise constant (and close to indicator vectors).
- Thinking of the rows of the solution matrices as another representation of data points, *K*-means clustering is a way of finding an appropriate group for each point.



Random Walks Point of View

- Tradition Probability Matrix

$$P_{ij} = \frac{W_{ij}}{\sum_k W_{ik}} \quad \Rightarrow \quad P = D^{-1}W$$



$$P = \begin{pmatrix} 0 & 1/4 & 1/4 & 0 & 0 & 0 \\ 1/2 & 0 & 1/4 & 1 & 1/3 & 0 \\ 1/2 & 1/4 & 0 & 0 & 1/3 & 1/2 \\ 0 & 1/4 & 0 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 0 & 0 & 1/2 \\ 0 & 0 & 1/4 & 0 & 1/3 & 0 \end{pmatrix}$$

Assume the weights of edges are 1.

- If the graph is connected and non-bipartite, then the random walk always possesses a unique stationary distribution π

$$\pi^T P = \pi^T \qquad \pi = [\,\pi_1, \cdots, \pi_n\,]^T$$

Random Walks Point of View

- Relationship between *Ncut* problem and random walks
  - For the random walk X_0 in the stationary distribution,

$$P(B \mid A) = P(X_1 \in B \mid X_0 \in A) \quad (A \cap B = \phi)$$

  - The problem of finding a partition such that random walk does not have many opportunities to jump between clusters is equivalent to *Ncut* problem due to:

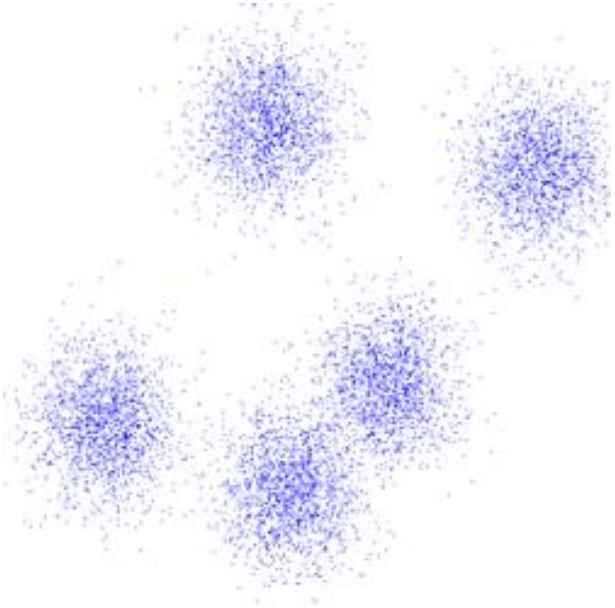
$$\mathrm{NCut}(A, \overline{A}) = P(\overline{A} \mid A) + P(A \mid \overline{A})$$

- Relationship between L_{rw} and P

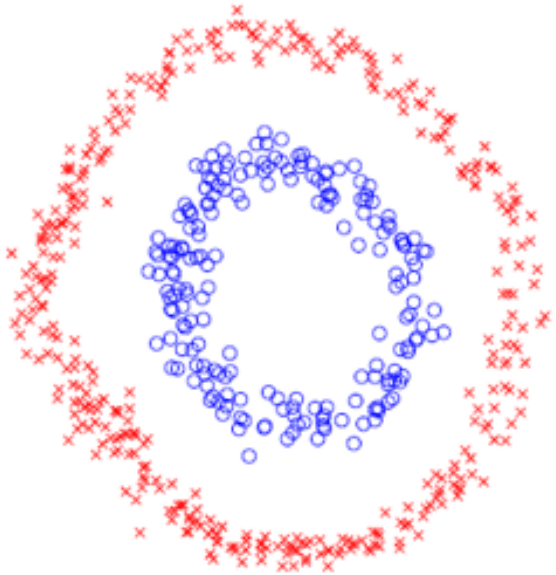
$$Lx = \lambda Dx \quad \Leftrightarrow \quad Px = (1 - \lambda)x$$

Spectral clustering v.s. *K*-means

- Two different criteria
  - Compactness, e.g. K-means, GMM
  - Connectivity, e.g. spectral clustering

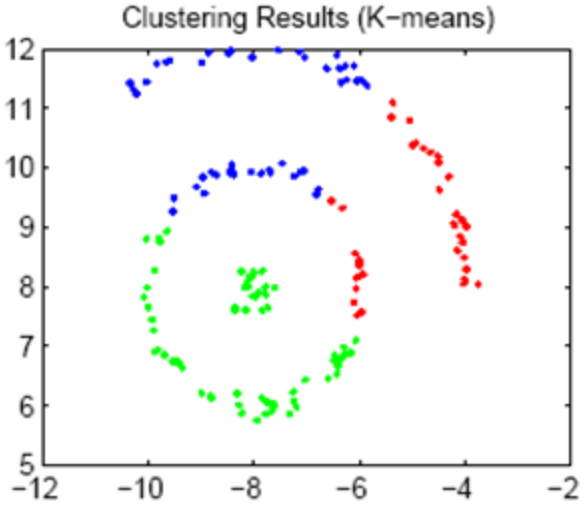


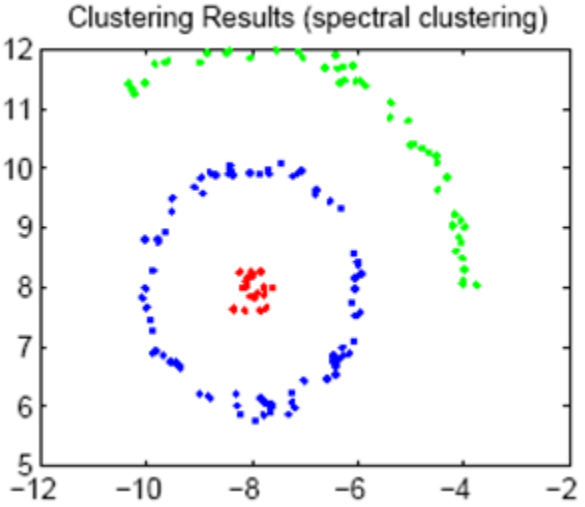
Compactness



Connectivity

Spectral clustering v.s. *K*-means





Spectral clustering v.s. *K*-means

从优化目标的角度来看：

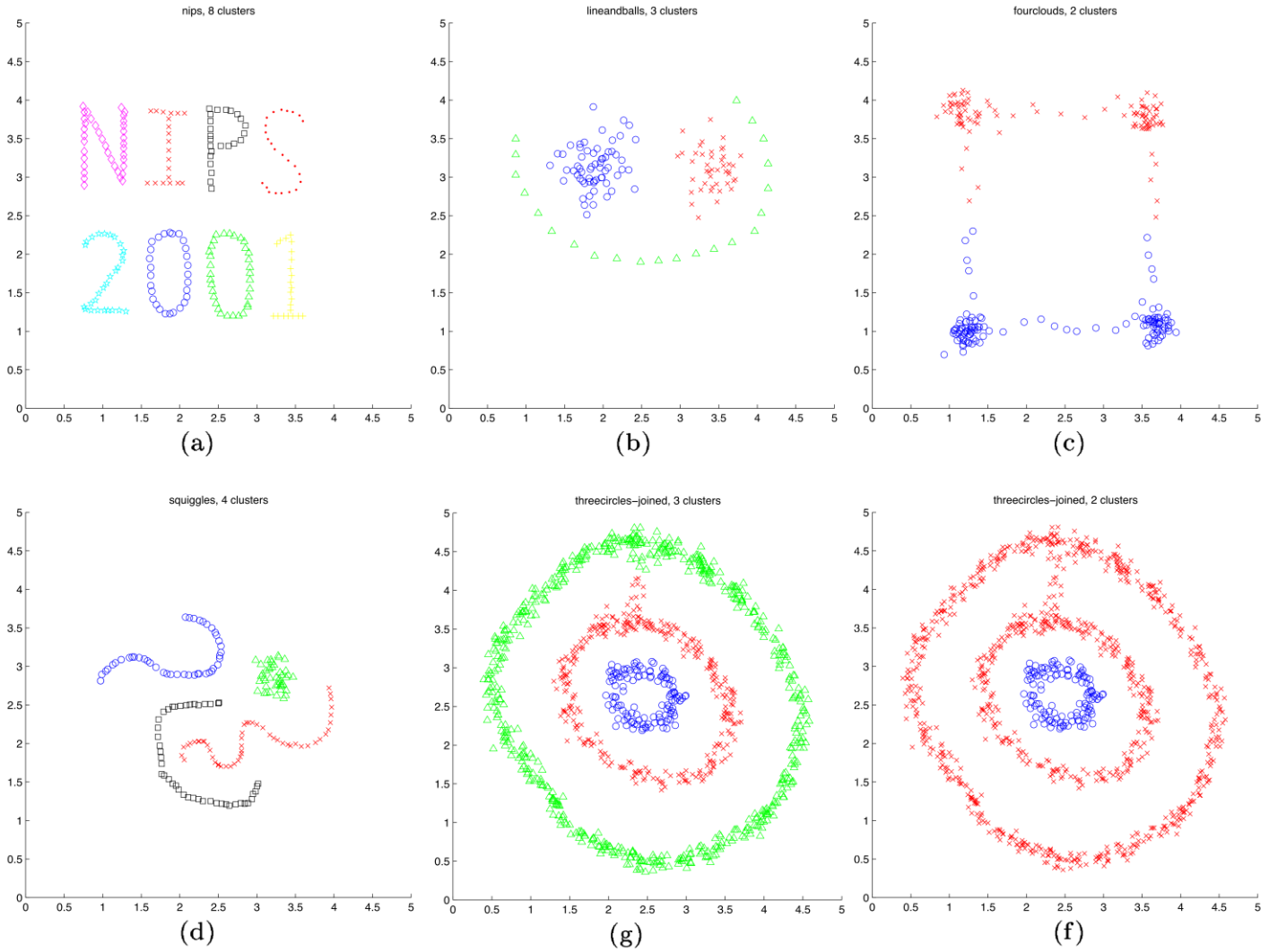
1. K-means致力于最小化类内的平均距离
2. 谱聚类致力于最小化类间的相似度

从图的观点来看：

local v.s. global

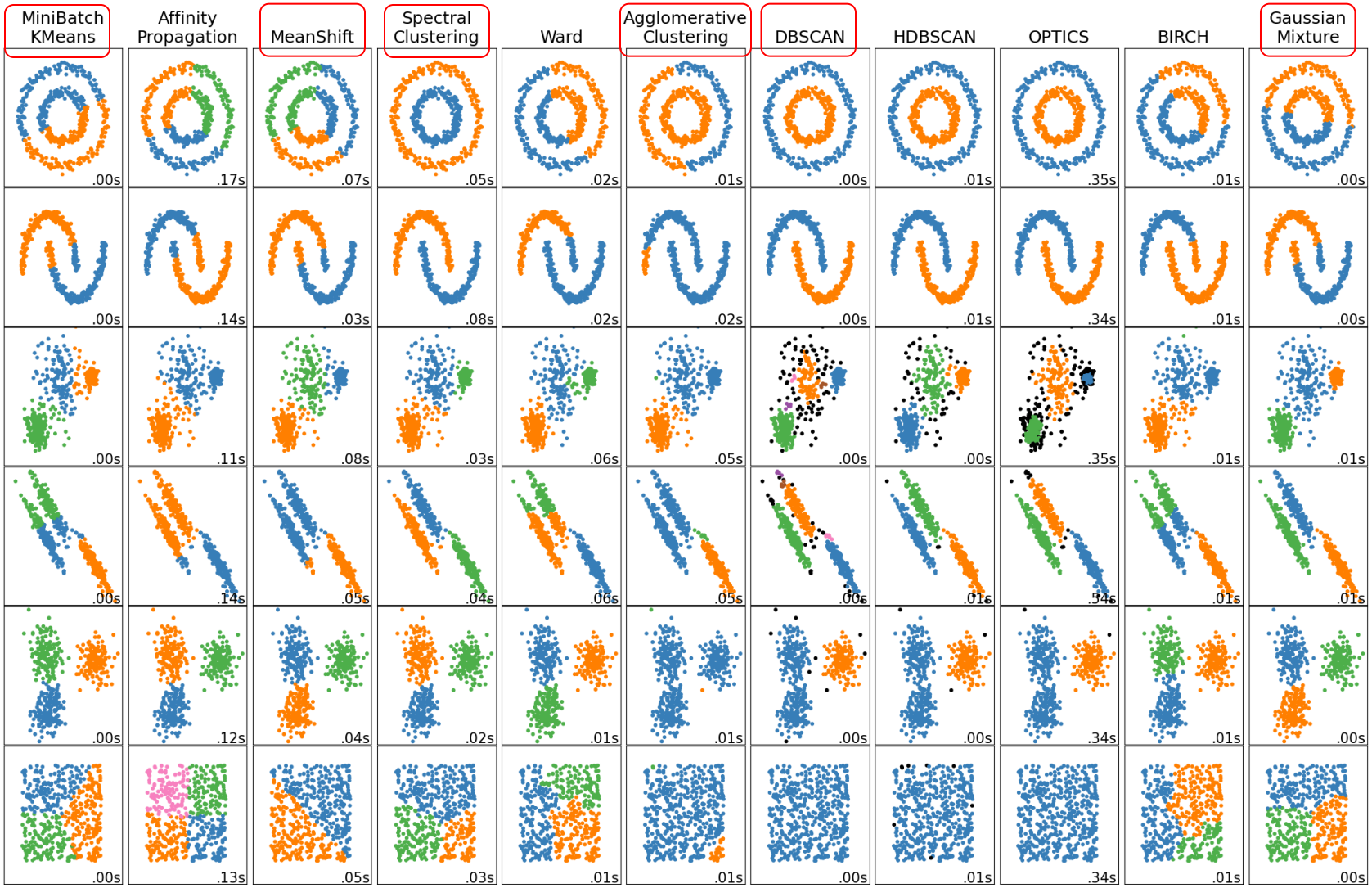
1. K-means采用的是全图
   即任意两个样本之间的距离都没有被忽略
2. 谱聚类采用的是近邻图
   一般只计算每个样本与其近邻之间的相似度

Toy examples



[Figures from Ng, Jordan, Weiss NIPS '01]

Toy examples



谱聚类应用

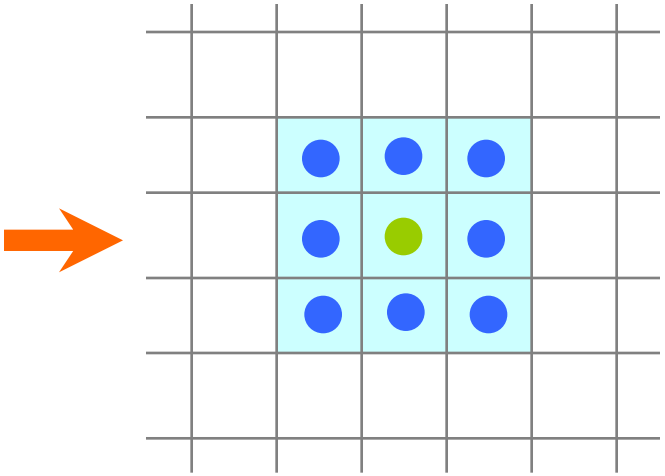
- 采用谱聚类将图像的前景目标分割出来

如何构造近邻图？

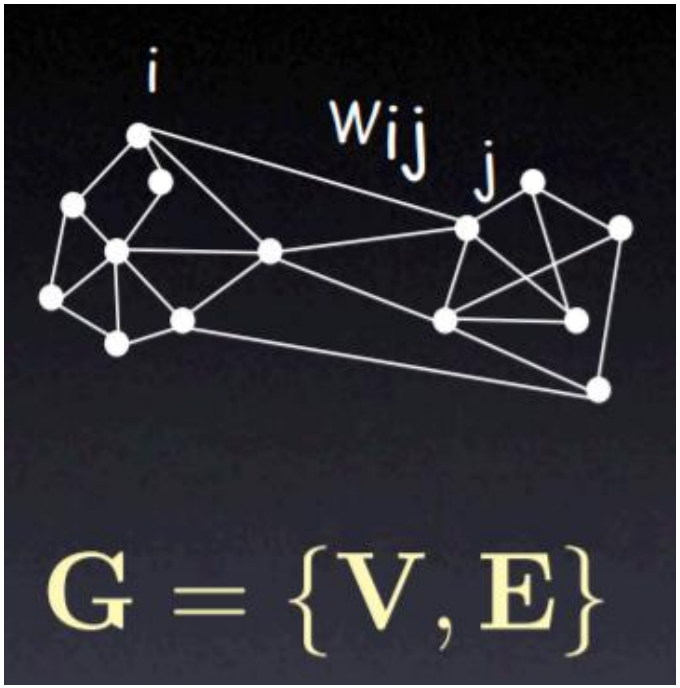
谱聚类应用

- 采用谱聚类将图像的前景目标分割出来
  - 以每个像素为一个顶点，以3X3为一个基本邻域，连接各像素，构建一个图。

图像



格子图



V: graph nodes

E: edges connection nodes

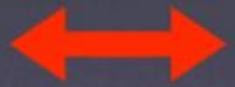
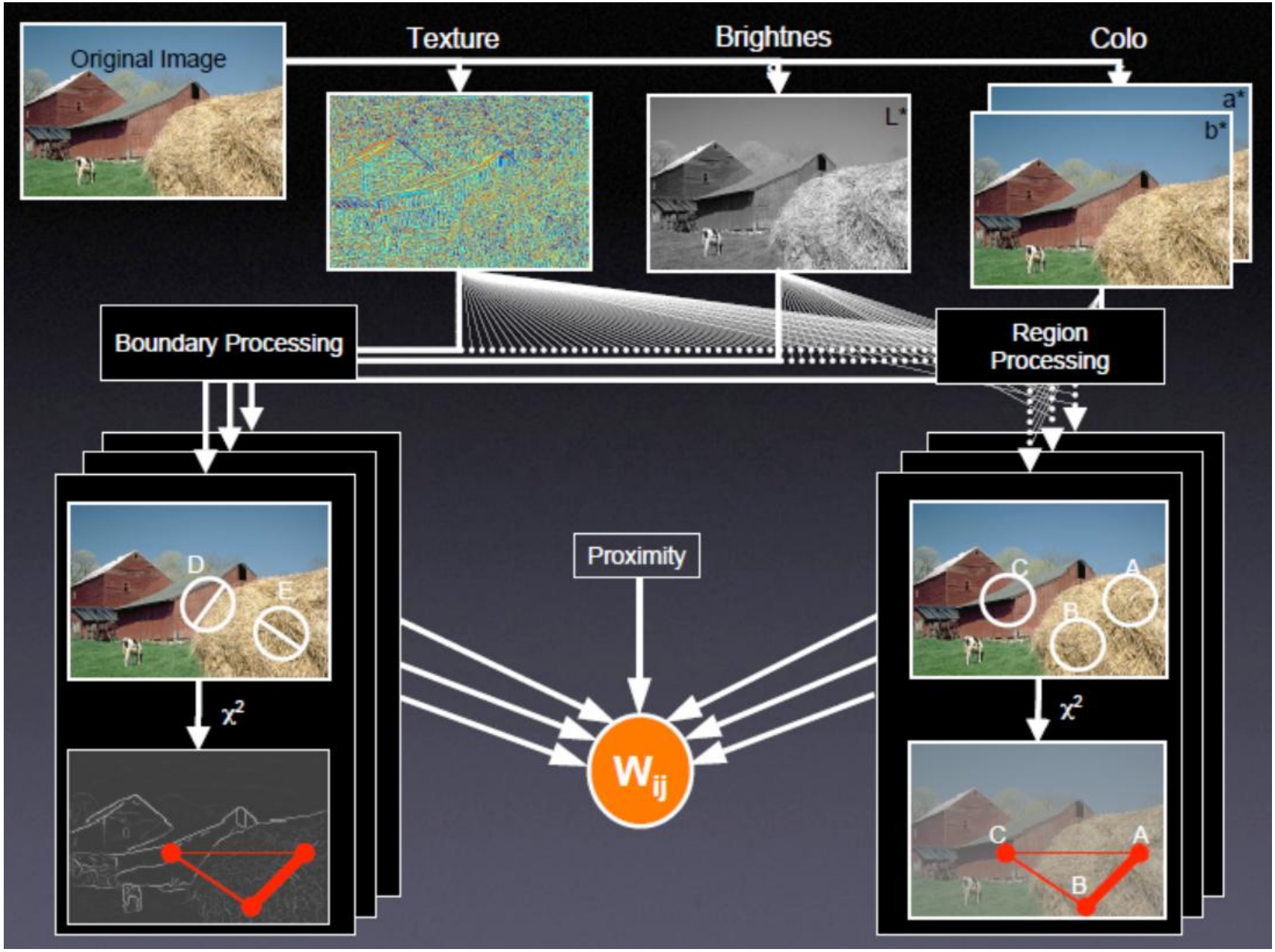


Image = { pixels }

Pixel similarity


Original Image
Texture
Brightnes
Colo
L*
a*
b*
Boundary Processing
Region Processing
D
E
Proximity
C
A
B
χ^2
χ^2
W_{ij}
C
A
B

存在的问题

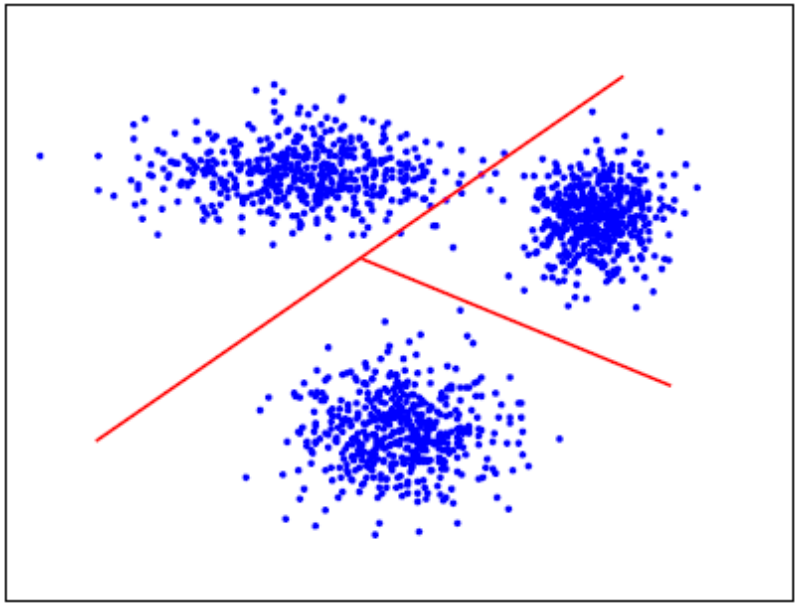
- **算法细节**
  - 核心问题是图构造
    - **局部连接 k 近邻 (ε-半径) 取多大？**
    - **点对权值如何计算？**
  - 特征值分解问题：对于超大矩阵，计算仍然不稳定，可能会导致结果很差。
    - To handle large data sets
    - Spectral clustering on reduced set selected by sampling or quantization
  - 采用 K-means 聚类也可能会影响聚类结果。
  - 聚类数目是一个开放问题。
    - Eigenvalue gap: choose the number k such that the gap between λ_k and λ_{k+1} is big

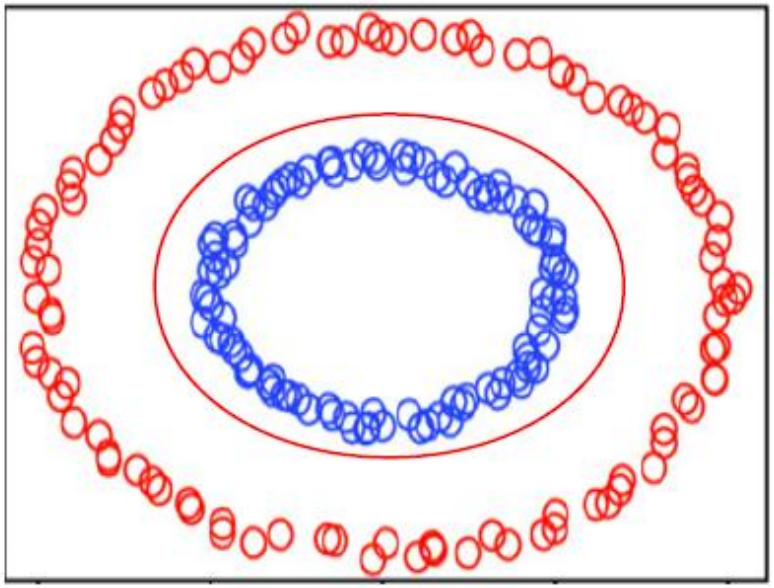
Kernel Clustering

核聚类

→ *K*-means in kernel space

Motivation

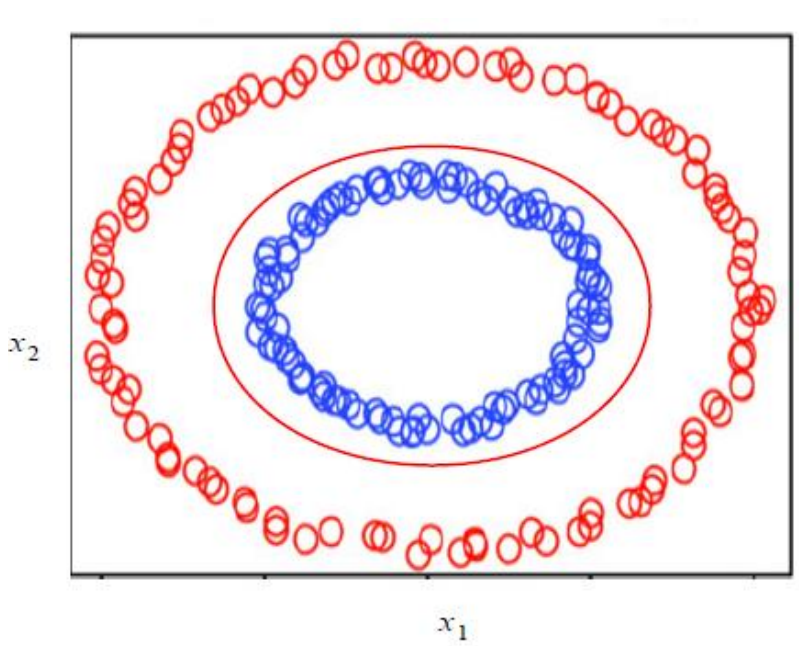


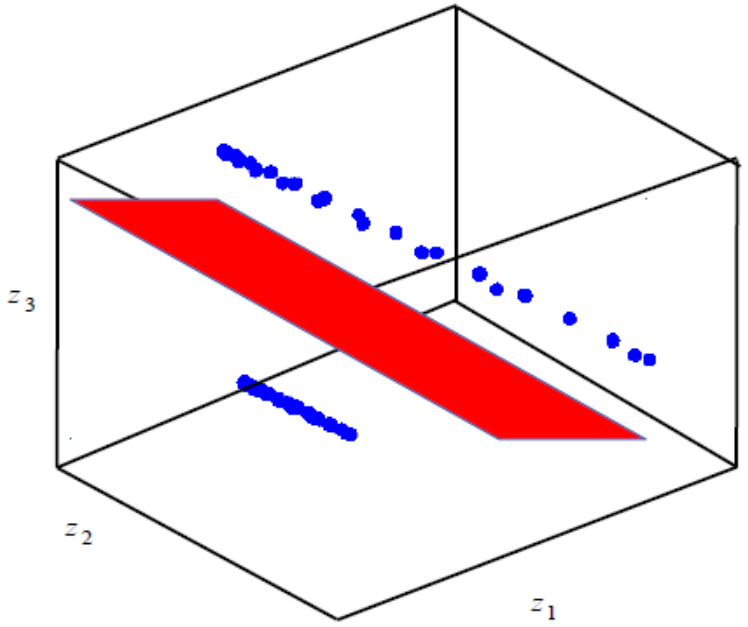


Euclidean distance assumes unit covariance and linear separability

Motivation

Any data set is linearly separable in sufficiently high dimensional space





$$\varphi : X \rightarrow H$$

Polynomial map: $\varphi(\mathbf{x}) = \varphi([x_1, x_2]) = [x_1^2, \sqrt{2}x_1x_2, x_2^2]$

What is kernel?

- Kernel function: inner product of the feature maps

$$\kappa(\mathbf{x},\mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$$

  - Example: polynomial map

$$\varphi([x_1, x_2]) = [x_1^2, \sqrt{2}x_1x_2, x_2^2]$$

$$\kappa(\mathbf{x},\mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y}) = [x_1^2, \sqrt{2}x_1x_2, x_2^2] \begin{bmatrix} y_1^2 \\ \sqrt{2}y_1y_2 \\ y_2^2 \end{bmatrix} = (\mathbf{x}^T\mathbf{y})^2$$

- Euclidean distance in H

$$\begin{aligned} \|\varphi(\mathbf{x}) - \varphi(\mathbf{y})\|_2^2 &= \varphi(\mathbf{x})^T\varphi(\mathbf{x}) - 2\varphi(\mathbf{x})^T\varphi(\mathbf{y}) + \varphi(\mathbf{y})^T\varphi(\mathbf{y}) \\ &= \kappa(\mathbf{x},\mathbf{x}) - 2\kappa(\mathbf{x},\mathbf{y}) + \kappa(\mathbf{y},\mathbf{y}) \end{aligned}$$

Different kernels

Polynomial (Linear)

$$\kappa(\mathbf{x},\mathbf{y}) = (\mathbf{x}^T\mathbf{y})^p$$

Gaussian

$$\kappa(\mathbf{x},\mathbf{y}) = \exp(-\lambda\|\mathbf{x}-\mathbf{y}\|_2^2)$$

Chi-square

$$\kappa(x, y) = 1 - \sum_{i=1}^{d} \frac{(x_i - y_i)^2}{0.5(x_i + y_i)}$$

Histogram intersection

$$\kappa(x, y) = \sum_{i=1}^{m} \min(\text{hist}(x), \text{hist}(y))$$

Kernel *K*-means

- Find clusters in the feature space H by minimize:

$$L(\boldsymbol{\mu}, \mathbf{X}, \mathbf{Z}) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \left\| \varphi(\mathbf{x}_n) - \boldsymbol{\mu}_k \right\|^2$$

  - Core technique

$$\begin{aligned} \left\| \varphi(\mathbf{x}) - \boldsymbol{\mu}_k \right\|^2 &= \varphi(\mathbf{x})^T \varphi(\mathbf{x}) - 2\varphi(\mathbf{x})^T \boldsymbol{\mu}_k + \boldsymbol{\mu}_k^T \boldsymbol{\mu}_k \\ &= \varphi(\mathbf{x})^T \varphi(\mathbf{x}) - 2\varphi(\mathbf{x})^T \frac{1}{|\mathcal{C}_k|} \sum_{\mathbf{y} \in \mathcal{C}_k} \varphi(\mathbf{y}) + \left(\frac{1}{|\mathcal{C}_k|} \sum_{\mathbf{y} \in \mathcal{C}_k} \varphi(\mathbf{y}) \right)^T \left(\frac{1}{|\mathcal{C}_k|} \sum_{\mathbf{y} \in \mathcal{C}_k} \varphi(\mathbf{y}) \right) \\ &= \kappa(\mathbf{x}, \mathbf{x}) - \frac{2}{|\mathcal{C}_k|} \sum_{\mathbf{y} \in \mathcal{C}_k} \kappa(\mathbf{x}, \mathbf{y}) + \frac{1}{|\mathcal{C}_k|^2} \sum_{\mathbf{y}, \mathbf{z} \in \mathcal{C}_k} \kappa(\mathbf{y}, \mathbf{z}) \end{aligned}$$

$$\boldsymbol{\mu}_k = \frac{1}{|\mathcal{C}_k|} \sum_{\mathbf{y} \in \mathcal{C}_k} \varphi(\mathbf{y})$$

Kernel K-means

- Input: $X=\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$, kernel function $\kappa(\cdot,\cdot)$, number of clusters K

1 Initialize K clusters: $\mathcal{C}_1, \mathcal{C}_2, \ldots, \mathcal{C}_K$

2 Set $t = 0$

3 For each point $\mathbf{x} \in X$ find its new cluster index by:

$$k^*(\mathbf{x}) = \arg\min_{k \in \{1,\ldots,K\}} \left\| \varphi(\mathbf{x}) - \boldsymbol{\mu}_k \right\|^2$$

4 Compute the updated clusters by:

$$\mathcal{C}_k = \{\mathbf{x} : k^*(\mathbf{x}) = k\}$$

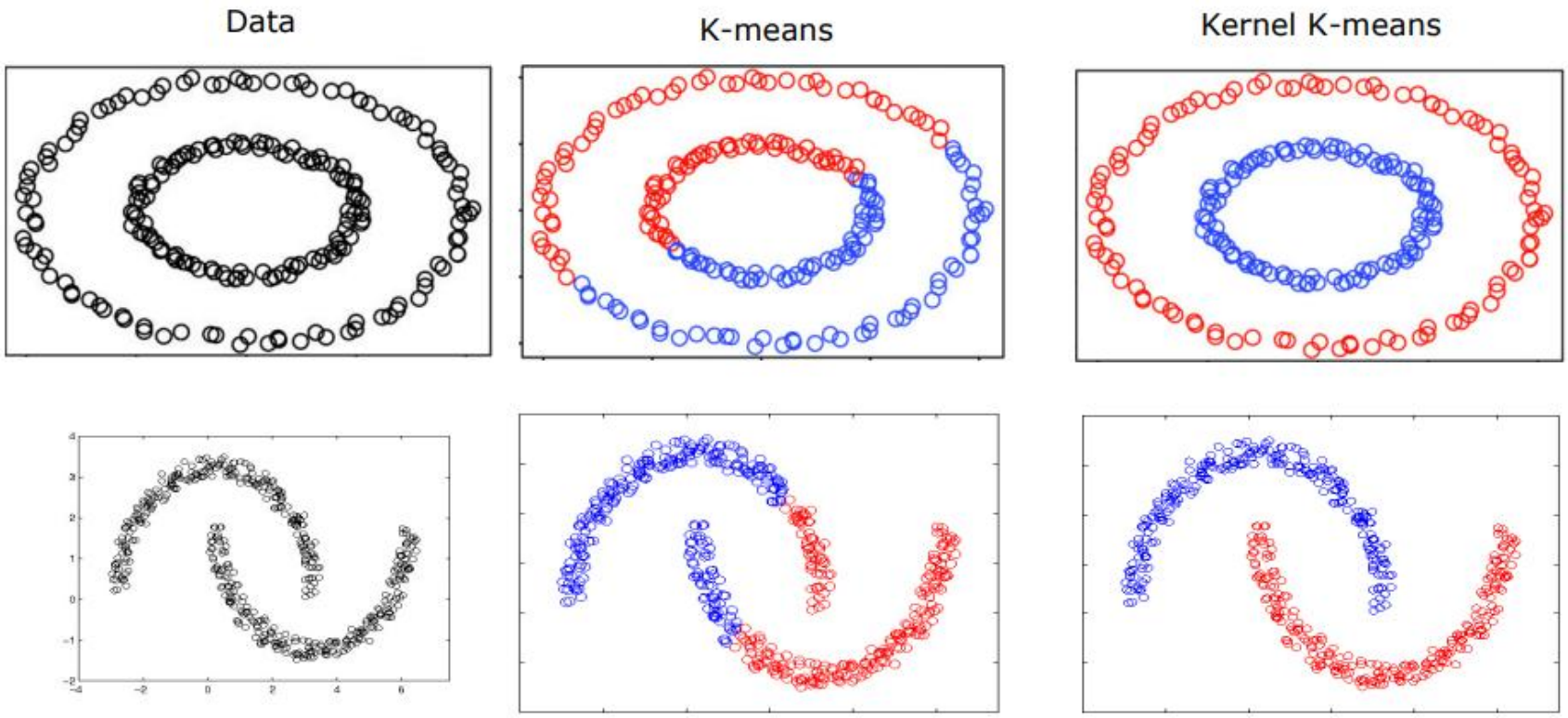
5 If not converged

set $t=t+1$, and go to Step 3;

else

output $\mathcal{C}_1, \mathcal{C}_2, \ldots, \mathcal{C}_K$

K-means v.s. Kernel *K*-means



Kernel K-means is able to find "complex" clusters.

Kernel *K*-means Challenges [以下略]

Scalability

- $O(n^2)$ complexity to calculate kernel
- More expensive than K-means

Out-of-sample clustering

- No explicit representation for the centers.
- Expensive to assign a new point to a cluster.

Kernel selection

- The best kernel is application and data-dependent
- Wrong kernel can lead to results worse than *K*-means

Scalability

No. of Objects (n)	No. of operations	
	K-means	Kernel K-means
	O(nCd)	$O(n^2C)$
1M	10^{11} (3.2)	10^{15}
10M	10^{12} (34.9)	10^{17}
100M	10^{13}(5508.5)	10^{19}
1B	10^{14}	10^{21}

d = 100; C=10

No. of Objects (n)	No. of operations	
	K-means	Kernel K-means
	O(nCd)	$O(n^2C)$
1M	10^{13} (6412.9)	10^{16}
10M	10^{14}	10^{18}
100M	10^{15}	10^{20}
1B	10^{16}	10^{22}

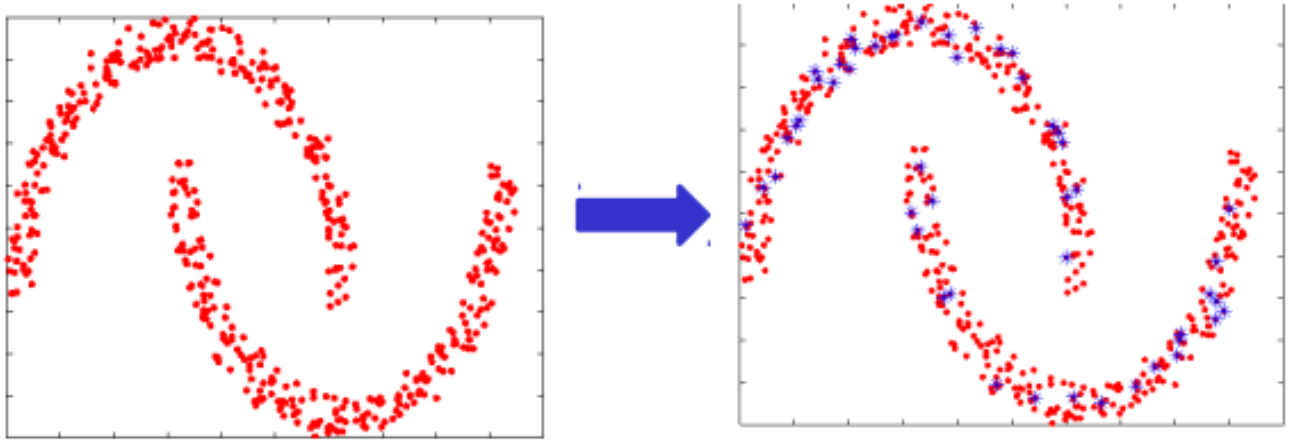
d = 10,000; C=10

* Runtime in seconds on Intel Xeon 2.8 GHz processor using 12 GB memory

Sampling based approximations

- Kernel matrix approximation
- Non-linear Random Projections

Approx. Kernel *K*-means



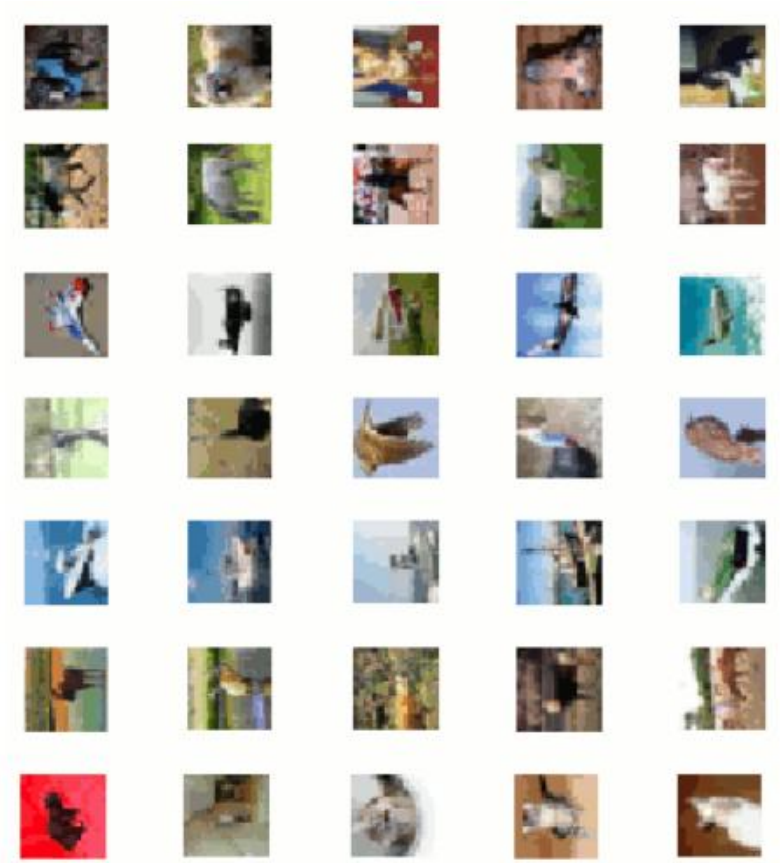
Randomly sample m points $\{y_1, y_2, \ldots y_m\}, m \ll n$ and compute the kernel similarity matrices

K_A (m x m) and K_B (n x m)

$$(K_A)_{ij} = \varphi(y_i)^T \varphi(y_j) \qquad (K_B)_{ij} = \varphi(x_i)^T \varphi(y_j)$$

Equivalent to running K-means on $K_B K_A^{-1} K_B^T$ Nystrom approximation

Clustering 80M Tiny Images



Example Clusters

Average clustering time into 100 clusters	
Approximate kernel K-means (m=1,000)	8.5 hours
K-means	6 hours

2.4 Ghz, 150 GB memory

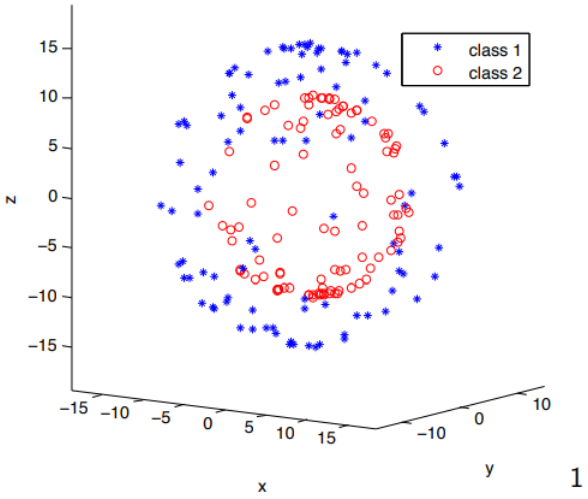
Clustering accuracy on CIFAR-10	
Kernel K-means	29.94
Approximate kernel K-means (m = 5,000)	29.76
Nystrom approximation based spectral clustering**	27.09
K-means	26.70

Kernel PCA + K-means

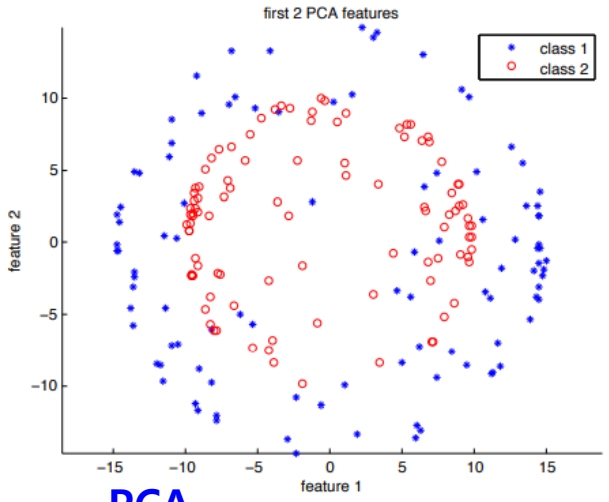
PCA: Eigen decomposition of **covariance** matrix

v.s.

KPCA: Eigen decomposition of **normalized kernel** matrix

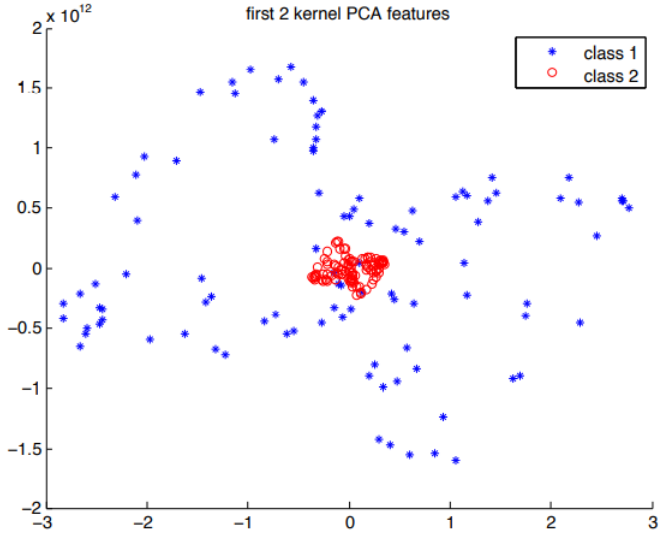


1



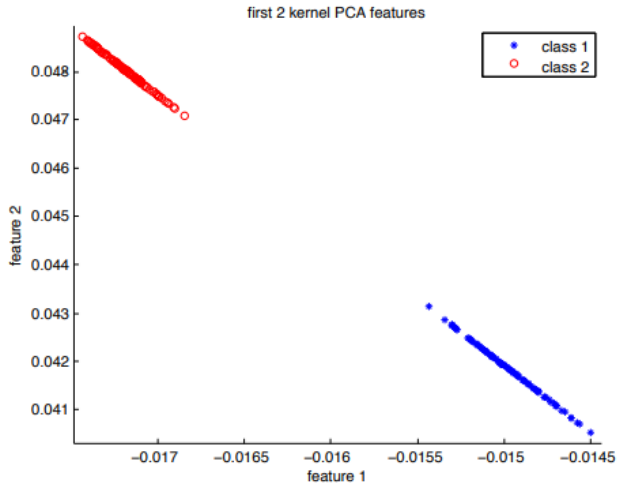
PCA

2



Polynomial Kernel ($d = 5$)

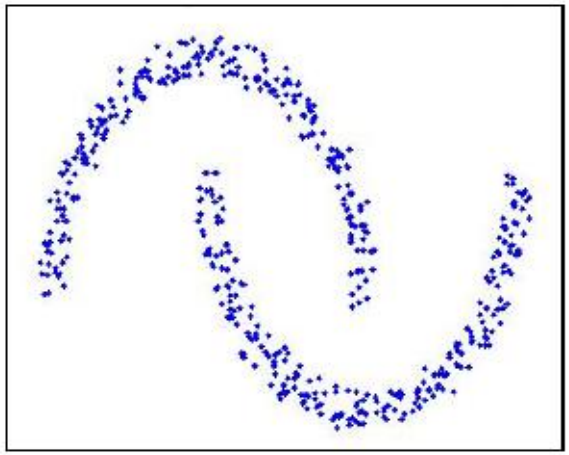
3

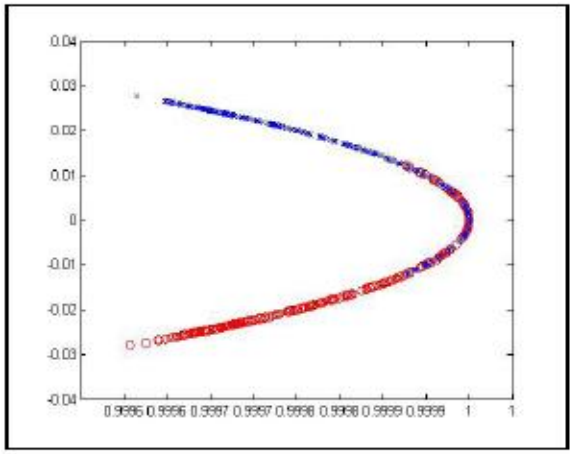


Gaussian Kernel ($\sigma = 20$)

4

Other Feature Space ?





Efficient Kernel Clustering Using **Random Fourier Features**, ICDM 2012

Randomly sample m vectors $\{\omega_1, \omega_2, \ldots \omega_m\}, m \ll n$

$$z(x) = \frac{1}{\sqrt{m}}\left[\cos\left(\omega_1^T x\right) \ldots \cos\left(\omega_m^T x\right) \ldots \sin\left(\omega_1^T x\right) \sin\left(\omega_m^T x\right)\right]$$

$$H = \left[z(x_1)^T \quad z(x_2)^T \ldots z(x_n)^T\right]$$

Cluster H using K-means and obtain the partition

Deep Clustering

深度聚类

→ *K*-means in deep space

Clustering-style Self-Supervised Learning

Mathilde Caron - FAIR Paris & Inria Grenoble

June 20th, 2021

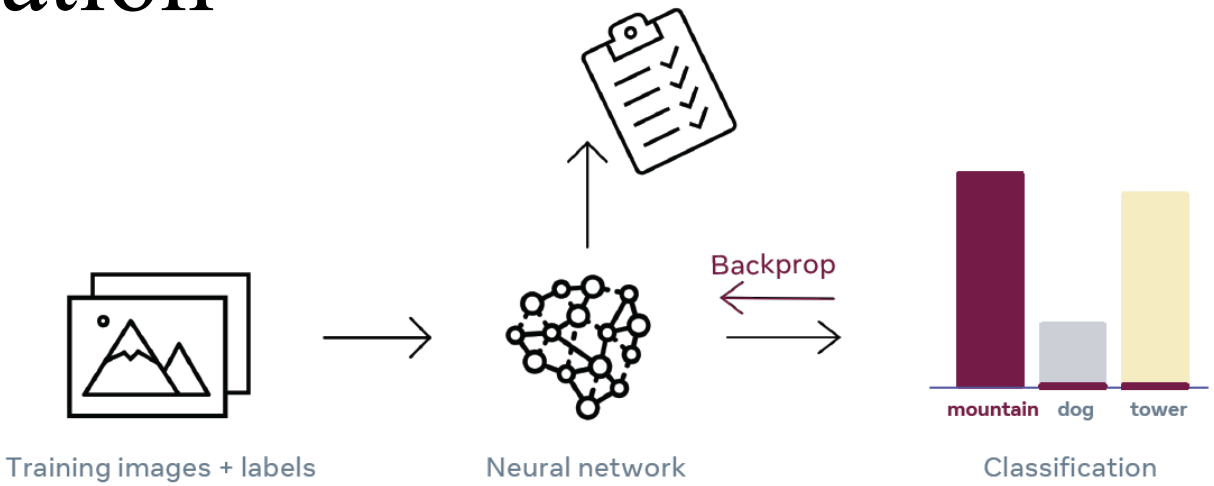
CVPR 2021 Tutorial:

Leave Those Nets Alone:

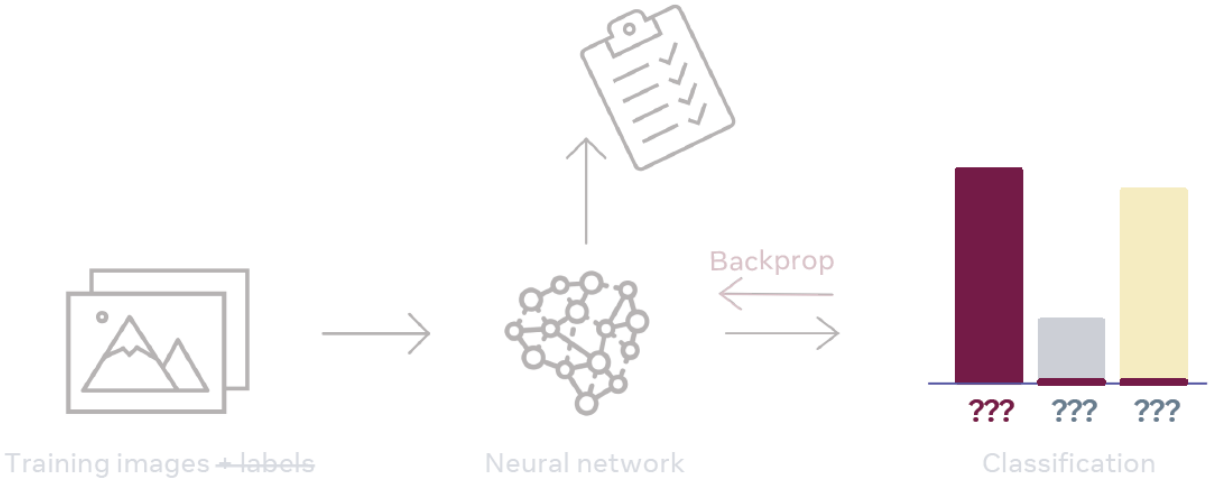
Advances in Self-Supervised Learning

FACEBOOK AI

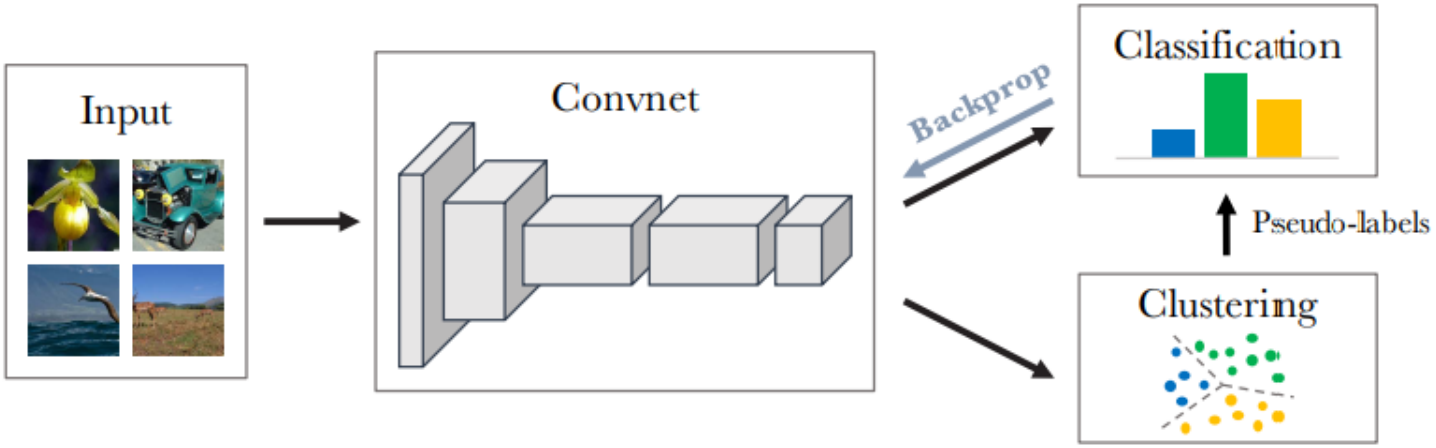
Motivation



Can we replace labels with clustering ?



DeepCluster: Deep Clustering for Unsupervised Learning of Visual Features (ECCV 2018)



We iteratively cluster deep features (*K*-means) and use the cluster assignments as pseudo-labels to learn the parameters of the convnet.

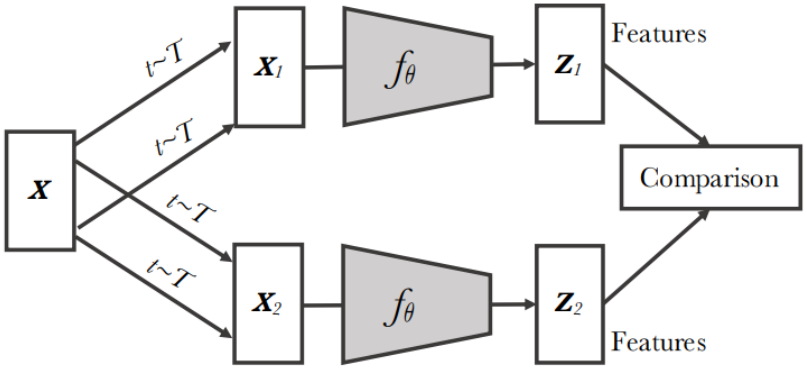
Problem 1: empty cluster

When a cluster becomes empty, we randomly select a non-empty cluster and use its centroid with a small random perturbation as the new centroid for the empty cluster. We then reassign the points belonging to the non-empty cluster to the two resulting clusters.

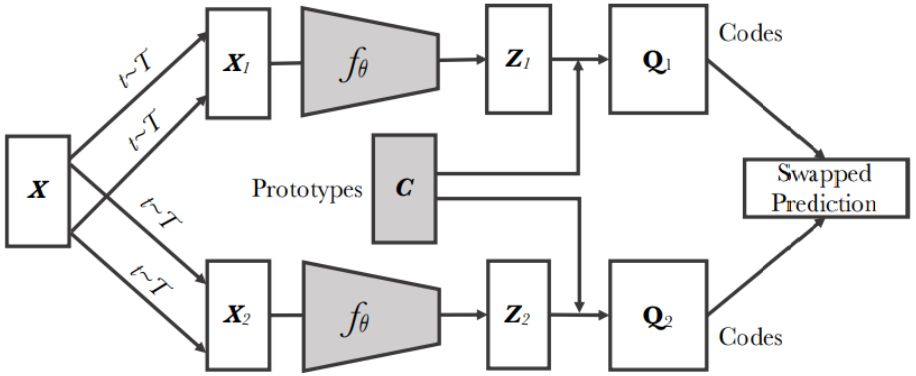
Problem 2: trivial parametrization: predict same output for every input

A strategy to circumvent this issue is to sample images based on a uniform distribution over the classes, or pseudo-labels. This is equivalent to weight the contribution of an input to the loss function by the inverse of the size of its assigned cluster.

SwAV: Unsupervised Learning of Visual Features by Contrasting Cluster Assignments (NeurIPS 2020)



Contrastive instance learning



Swapping Assignments between Views (Ours)

Contrastive instance learning (left)

- features from different transformations of the same image are compared to each other.

SwAV (right)

- first obtain "codes" by assigning features to prototype vectors.
- then solve a "swapped" prediction problem.
- does not directly compare image features.
- prototype vectors are learned along with the ConvNet.

SwAV: Unsupervised Learning of Visual Features by Contrasting Cluster Assignments (NeurIPS 2020)

Method	Arch.	Param.	Top1
Supervised	R50	24	76.5
Colorization [65]	R50	24	39.6
Jigsaw [46]	R50	24	45.7
NPID [58]	R50	24	54.0
BigBiGAN [15]	R50	24	56.6
LA [68]	R50	24	58.8
NPID++ [44]	R50	24	59.0
MoCo [24]	R50	24	60.6
SeLa [2]	R50	24	61.5
PIRL [44]	R50	24	63.6
CPC v2 [28]	R50	24	63.8
PCL [37]	R50	24	65.9
SimCLR [10]	R50	24	70.0
MoCov2 [11]	R50	24	71.1
SwAV	R50	24	**75.3**

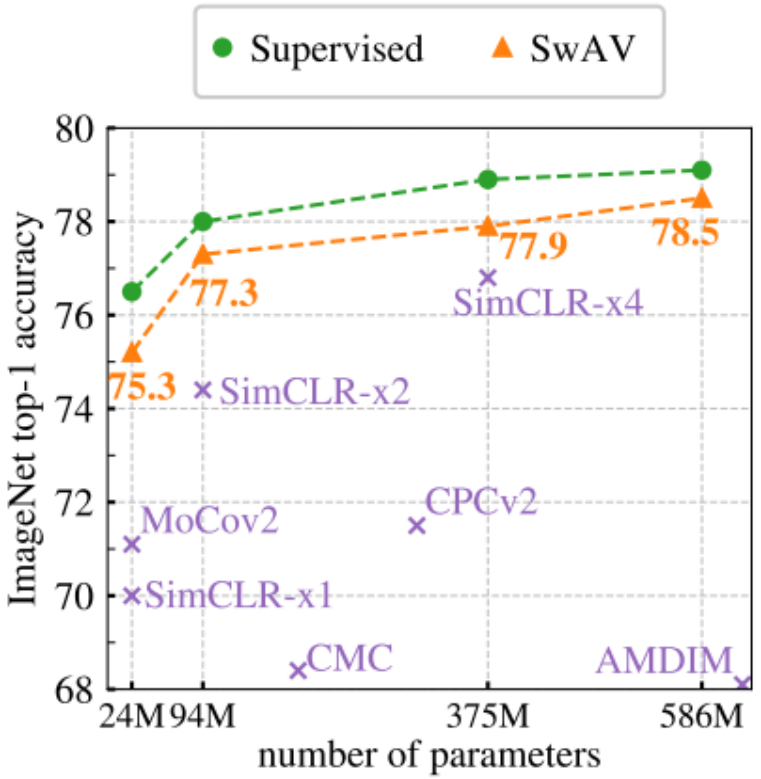


Figure 2: **Linear classification on ImageNet.** Top-1 accuracy for linear models trained on frozen features from different self-supervised methods. **(left)** Performance with a standard ResNet-50. **(right)** Performance as we multiply the width of a ResNet-50 by a factor $\times 2$, $\times 4$, and $\times 5$.

总结与讨论

- K-means clustering
  - K-medians, Competitive learning
- Mean-shift clustering, DBSCAN
- Gaussian Mixture Model
- Hierarchical Clustering
- Spectral clustering
- Kernel clustering
- Deep clustering

Thank All of You!

(Questions?)